



UNIVERSITÀ DEGLI STUDI DI PADOVA

Corso di Laurea in Informatica
Insegnamento di Ingegneria del Software

Anno Accademico 2025/2026

Specifica Tecnica

Architettura e Design di Sistema

Gruppo: Synergo Unit

Email: synergounit20@gmail.com

Versione: **v1.1.0**
Data: **2026-08-24**

Registro delle Modifiche

Versione	Data	Autore	Verificatore	Descrizione
1.1.0	2026-08-24	A. Ricardo Rupì	Sofia De Blasi	Revisione sezioni e integrazione di nuovi flussi di dati.
1.0.0	2026-08-10	Francesco Marcon		Approvazione ingresso in baseline PB.
0.4.0	2026-08-09	Sofia De Blasi	Francesco Marcon	Stesura sezione Requisiti di Sistema.
0.3.0	2026-08-08	Sofia De Blasi	Francesco Marcon	Stesura sezioni Interfaccia API, Design Pattern Utilizzati, Flusso dei Dati e Tracciamento dei Requisiti con diagrammi.
0.2.0	2026-08-08	Sofia De Blasi	Francesco Marcon	Stesura sezioni Tecnologie e Architettura con diagrammi.
0.1.0	2026-08-06	Sofia De Blasi	Francesco Marcon	Stesura Introduzione.
0.0.0	2026-08-06	Sofia De Blasi	Francesco Marcon	Prima stesura della struttura del documento.

Indice

1	Introduzione	6
1.1	Scopo del documento	6
1.2	Scopo del prodotto	6
1.3	Glossario	6
1.4	Riferimenti	7
1.4.1	Normativi	7
1.4.2	Informativi	7
2	Tecnologie Utilizzate	8
2.1	Linguaggi, runtime e framework _G	8
2.2	Librerie Frontend	8
2.3	Librerie Backend	9
2.4	Persistenza dei dati	10
2.5	Testing	10
2.6	Infrastruttura e strumenti di sviluppo	10
2.7	Motivazioni delle scelte tecnologiche	11
3	Architettura del Sistema	12
3.1	Architettura Generale	12
3.2	Architettura di Deployment	12
3.3	Architettura del Frontend	13
3.3.1	Gestione dello Stato e dell'Editor	14
3.3.2	Gestione Interazioni AI (Widget/Pannello AI)	15
3.4	Architettura del Backend	16
3.4.1	Livello di Presentazione	17
3.4.2	Livello di Business Logic (Core LLM)	18
3.4.3	Componente di Esportazione (Export)	19
4	Decomposizione del Sistema	20
5	Interfaccia API e Comunicazione	22
5.1	Struttura dei Payload AI	22
5.2	Gestione degli Errori e Fallback _G	23
6	Design Pattern Utilizzati	24
6.1	Pattern Creazionali	24
6.1.1	Simple Factory	24

6.1.2	Builder	24
6.2	Pattern Strutturali	25
6.2.1	Decorator	25
6.2.2	Adapter _G	25
6.2.3	Proxy (remoto)	26
6.2.4	Template Method	27
6.3	Pattern Comportamentali	27
6.3.1	Strategy	27
6.3.2	Command	28
6.3.3	Observer _G	28
6.3.4	Dependency Injection	29
7	Flusso dei Dati (Data Flow)	31
7.1	Elaborazione di un'operazione AI	31
7.2	Gestione della proposta AI ed elenco delle operazioni	33
7.3	Editing del testo e Undo/Redo	36
7.4	Persistenza locale della nota	40
7.5	Esportazione	43
8	Tracciamento e Stato dei Requisiti	45
8.1	Tabella Tracciamento Componenti-Requisiti	45
8.2	Stato di Soddisfacimento dei Requisiti	46
9	Requisiti di Sistema	50
9.1	Hardware	50
9.2	Software	50
9.3	Network	51

Elenco delle figure

1	Diagramma dei componenti del sistema.	12
2	Diagramma UML di Deployment.	13
3	Diagramma dei package del Frontend.	14
4	Diagramma delle classi: <code>MarkdownContentEditor</code> e la famiglia di comandi di editing (pattern Command).	14
5	Diagramma delle classi: architettura MVC pull del Frontend (editing e interazioni AI) e rappresentazione dello stato di <code>AIRequestModel</code> come union discriminata. .	15
6	Diagramma delle classi: interfacce Proxy verso i servizi Backend (<code>NoteService</code> , <code>ExportService</code> , <code>AIService</code>) e relative implementazioni.	16
7	Diagramma dei package del Backend.	17
8	Diagramma delle classi del livello di Presentazione e DI del Backend.	18
9	Diagramma delle classi del dominio AI: operazioni, costruzione dei prompt e integrazione con il servizio LLM.	19
10	Diagramma delle classi del package Export (Template Method).	19
11	Richiesta operazione AI (tratto Frontend).	32
12	Richiesta AI (tratto REST→LLM): attraversa in un unico flusso i pattern Strategy, Builder, Simple Factory, Decorator e Adapter.	32
13	Ricezione proposta AI (tratto Frontend).	33
14	Accetta proposta AI (riuso del pattern Command): <code>ReplaceContentCommand</code> nel ramo normale, <code>InsertTextCommand</code> come rete di sicurezza.	34
15	Rifiuta, Rigenera e Interrompi la proposta AI (frame <code>alt</code> a tre rami).	35
16	Elenco operazioni disponibili (Simple Factory self-registration).	36
17	Formattazione di un testo e relativo Undo.	37
18	Redo (continuazione di “Formattazione con Undo”).	37
19	Inserimento di una tabella, con validazione delle dimensioni.	38
20	Copia, Taglia e Incolla (frame <code>alt</code> a tre rami).	39
21	Salvataggio di una nota in locale.	40
22	Salvataggio con errore di I/O.	41
23	Importazione/Apertura di una nota esistente, con fallback per browser senza File System Access API.	42
24	Apertura nota con errore, con distinzione tra annullamento utente ed errore di I/O. .	43
25	Esportazione di una nota in PDF, dal click in <code>App.vue</code> al Template Method sul Backend.	44

Elenco delle tabelle

1	Linguaggi, runtime e framework	8
2	Librerie Frontend	8
3	Librerie Backend	9
4	Strumenti di Testing	10
5	Infrastruttura e strumenti di sviluppo	10
6	Componenti Architetture di Alto Livello	20
17	Tracciamento Componenti → Requisiti	45
18	Stato dei Requisiti	47

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di definire l'architettura di base e di dettaglio del sistema **Second Brain_G**. Vengono illustrate le scelte tecnologiche effettuate dal gruppo Synergo Unit durante la fase di Product Baseline_G, i pattern architetturali e di design adottati, la decomposizione in componenti del sistema e il tracciamento bidirezionale tra queste componenti e i requisiti individuati nella fase di Analisi.

A differenza della Technology Baseline_G, in cui l'obiettivo era esclusivamente validare la fattibilità tecnica delle scelte effettuate tramite un Proof of Concept_G, questo documento consolida un'architettura definitiva, coerente con il PoC_G ma estesa e rifinita per coprire la totalità dei requisiti obbligatori individuati in Analisi dei Requisiti_G (Sezione 4 - Requisiti).

1.2 Scopo del prodotto

Second Brain è un'applicazione web per la redazione di testi in formato Markdown_G, pensata per superare i limiti dei classici chatbot_G testuali fornendo un editor integrato in cui un Large Language Model_G (LLM_G) può attivamente elaborare, criticare e generare parti di testo su richiesta dell'utente.

Le funzionalità principali riguardano:

- la scrittura e formattazione_G di testo Markdown (grassetto, corsivo, sottolineato, barrato, citazioni, intestazioni, link, elenchi, tabelle) con anteprima in tempo reale e cronologia di annullamento/ripristino;
- il supporto AI alla scrittura: riassunto, traduzione_G (Inglese, Francese, Spagnolo, e Tedesco come estensione facoltativa) e riscrittura del testo selezionato;
- l'analisi critica del testo secondo il metodo dei “Sei Cappelli per Pensare_G” di Edward de Bono (dati e fatti, emozioni, rischi, opportunità, creatività, logica/controllo);
- il *Distant Writing_G*: la generazione di interi blocchi di testo a partire da un prompt_G fornito dall'utente in una finestra dedicata;
- la gestione dei dati: salvataggio, apertura ed esportazione (PDF, HTML_G, JSON_G) delle note come requisito obbligatorio basato sul file system_G locale, con gestione di note remote, autenticazione utente e persistenza server-side come estensione opzionale.

L'utente di riferimento è un individuo senza competenze tecniche pregresse (tipicamente uno studente) che utilizza l'applicazione per prendere appunti_G e desidera supporto nella loro elaborazione tramite LLM; l'interfaccia deve pertanto risultare semplice e intuitiva. L'unico attore_G secondario del sistema è il servizio LLM esterno, richiamato tramite API_G compatibile con lo standard OpenAI.

1.3 Glossario

Al fine di evitare ambiguità, i termini tecnici o con significato specifico sono seguiti dal pedice e definiti nel documento Glossario_G, v3.2.0

Ultimo accesso: 2026-08-27.

1.4 Riferimenti

1.4.1 Normativi

- Capitolato C6 – Second Brain (Zucchetti_G S.p.A.)
Ultimo accesso: 2026-08-06;
- Norme di Progetto_G, v3.0.0
Ultimo accesso: 2026-08-27;

1.4.2 Informativi

- Analisi dei Requisiti, v2.1.0, in particolare la Sezione 3 (Casi d'Uso_G) e la Sezione 4 (Requisiti)
Ultimo accesso: 2026-08-27;
- Analisi dello Stato dell'Arte, v1.0.0, in particolare la Sezione 2 (Tecnologie Frontend_G), la Sezione 3 (Tecnologie Backend_G) e la Sezione 4 (Struttura Tecnica Preliminare del Proof of Concept)
Ultimo accesso: 2026-08-27;
- Piano di Qualifica_G, v2.x.x (in aggiornamento continuo), in particolare la Sezione 4 (Specifiche dei Test)
Ultimo accesso: 2026-08-27;
- Vue.js Guide – Reactivity Fundamentals
Ultimo accesso: 2026-08-18;
- CodeMirror_G – System Guide
Ultimo accesso: 2026-08-18;
- FastAPI – Dependencies
Ultimo accesso: 2026-08-20;
- OpenAI API Reference, pagina *Create chat completion* – standard di riferimento per l'interazione con i modelli LLM (R2-V-O)
Ultimo accesso: 2026-08-17;
- Docker_G Docs – Docker Compose_G
Ultimo accesso: 2026-08-16;

2 Tecnologie Utilizzate

Le tecnologie adottate sono state selezionate secondo i criteri motivati in dettaglio in [Analisi dello Stato dell'Arte](#) (Sezioni 2–3), a cui si rimanda per l'analisi comparativa delle alternative scartate. Le versioni riportate corrispondono ai valori definiti nei file di lock del progetto (`package-lock.json`, `requirements.txt`) alla data di redazione di questo documento.

2.1 Linguaggi, runtime e framework

Tabella 1: Linguaggi, runtime e framework

Tecnologia	Versione	Descrizione
TypeScript	6.0.x	Linguaggio principale del Frontend; la tipizzazione statica riduce gli errori di integrazione con le interfacce dei payload AI.
Vue.js	3.5.x	Framework a componenti del Frontend, scelto per la progressività e la bassa curva di apprendimento (Sezione 2 di Analisi dello Stato dell'Arte).
Python	3.11	Linguaggio del Backend, scelto per la maturità del suo ecosistema nell'integrazione con strumenti di intelligenza artificiale e NLP per per la pregressa conoscenza da parte di alcuni membri del gruppo.
FastAPI	0.136.3	Framework Backend per API REST asincrone; espone i router <code>APIRouter</code> ed <code>ExportRouter</code> (Sezione 5).
Node.js	22.x	Funge da runtime di esecuzione essenziale per il server di sviluppo (Vite) e la build del codice TypeScript/Vue.js lato frontend.

2.2 Librerie Frontend

Tabella 2: Librerie Frontend

Libreria	Versione	Descrizione
CodeMirror	6.0.2	Componente editor di testo integrato nel Frontend (Sezione 2 di Analisi dello Stato dell'Arte); il nucleo (<code>codemirror</code>) è integrato da un insieme di pacchetti <code>@codemirror/*</code> ufficiali per le funzionalità specifiche richieste (riga sotto).
<code>@codemirror/state</code> , <code>@codemirror/view</code> , <code>@codemirror/commands</code>	6.x	Stato dell'editor, rendering e comandi base (selezione, cronologia nativa del browser) su cui si innesta la gestione applicativa di Undo/Redo (Sezione 6, pattern Command).
<code>@codemirror/lang-markdown</code> , <code>@codemirror/language-data</code>	6.x	Riconoscimento della sintassi Markdown e evidenziazione del testo nell'editor.
Continua nella pagina successiva...		

Tabella 2 – Continua dalla pagina precedente

Libreria	Versione	Descrizione
@codemirror/theme-one-dark	6.x	Tema visivo dell'editor.
vue-codemirror6	1.5.2	Binding Vue.js per l'integrazione di Code-Mirror come componente reattivo.
marked	18.0.4	Conversione del contenuto Markdown della nota in HTML per l'anteprima.
DOMPurify	3.2.7	Sanitizzazione dell'HTML generato da marked prima del rendering, a protezione da XSS.

2.3 Librerie Backend

Tabella 3: Librerie Backend

Libreria	Versione	Descrizione
Pydantic	2.13.4	Validazione _G dei dati e definizione tipizzata dei DTO _G delle operazioni AI (AIOperationRequest , AIOperationResponse , ExportRequest ; Sezione 5), integrata nativamente in FastAPI.
openai	2.41.0	Client ufficiale per l'API compatibile con lo standard OpenAI (R2-V-O, R3-V-O); configurato con base_url verso l'endpoint _G fornito dall'azienda proponente.
httpx	0.28.1	Libreria HTTP asincrona, dipendenza di openai per le chiamate al servizio LLM esterno.
Uvicorn	0.49.0	Server ASGI su cui viene eseguito il Backend FastAPI.
python-dotenv	1.2.2	Caricamento delle variabili d'ambiente (ZUCCHETTI_LLM_BASE_URL , ZUCCHETTI_LLM_API_KEY) da file .env , per non includere credenziali nel codice sorgente, coerentemente con la gestione lato Backend richiesta da R8-Q-O.
markdown-it-py	3.0.0	Libreria di terze parti utilizzata per effettuare il parsing del testo Markdown e generare l'output HTML omogeneo condiviso dai componenti di esportazione HTML e PDF.
mdit-py-plugins	0.6.1	Estensioni a markdown-it-py (es. tabelle) necessarie per coprire la sintassi Markdown supportata dall'editor in fase di esportazione.
linkify-it-py	2.1.0	Riconoscimento automatico dei link nel testo durante il parsing Markdown, dipendenza di markdown-it-py .
xhtml2pdf	0.2.17	Conversione dell'HTML prodotto da markdown-it-py in un documento PDF (R77-F-O); usata esclusivamente da PdfExporter .

2.4 Persistenza dei dati

Il gruppo ha scelto di non implementare alcun sistema di persistenza server-side: la gestione dei dati si basa esclusivamente sul file system locale del browser (salvataggio, apertura ed esportazione della nota, R75–R77), che copre i requisiti obbligatori del capitolato.

La persistenza remota delle note e la gestione di utenti autenticati sono funzionalità opzionali (R4-V-Op, R78–R86) che il gruppo ha deciso di non realizzare, per concentrare le risorse disponibili sul consolidamento dei requisiti obbligatori; non è pertanto presente alcun database nello stack tecnologico_G né nell'architettura di deployment (Sezione 3.2). Lo stato di questi requisiti opzionali è riportato in dettaglio in Sezione 8.

2.5 Testing

Tabella 4: Strumenti di Testing

Strumento	Descrizione
Vitest	Framework di test per il Frontend TypeScript/Vue.js; esegue sia i Test di Unità (<code>frontend/src/**</code>) che i Test di Sistema (<code>test.sistema/</code>), integrati nella stessa esecuzione tramite <code>test.include</code> di <code>vite.config.js</code> (Piano di Qualifica , Sezione 4).
@vue/test-utils	Libreria per il montaggio e l'interazione programmatica con i componenti Vue.js nei test, sia di unità che di sistema.
pytest, pytest-asyncio	Framework di test per il Backend Python; <code>pytest-asyncio</code> abilita il test degli endpoint asincroni di FastAPI.
pytest-cov, @vitest/coverage-v8	Calcolo della copertura di codice rispettivamente per Backend e Frontend, con soglie minime verificate in CI _G (~95% su Backend, soglia 90%; ~90% linee su Frontend, soglia 85%).

Il dettaglio della strategia di test per livello (unità, integrazione, sistema, accettazione) è descritto nel [Piano di Qualifica](#), Sezione 4.

2.6 Infrastruttura e strumenti di sviluppo

Tabella 5: Infrastruttura e strumenti di sviluppo

Strumento	Descrizione
Docker Compose	Orchestrazione dei container <code>frontend</code> e <code>backend</code> (Sezione 3.2). Il server di sviluppo Vite del container <code>frontend</code> inoltra le richieste verso <code>/api</code> al container <code>backend</code> (<code>http://backend:8000</code>).
Git e GitHub	Versionamento del codice e gestione collaborativa del repository _G , con tracciamento delle attività tramite issue _G (requisito R2-Q-O).
Continua nella pagina successiva...	

Tabella 5 – Continua dalla pagina precedente

Strumento	Descrizione
GitHub Actions _G	Automazione di build, esecuzione dei test e controlli di qualità a ogni push/pull request _G (requisito R1-Q-O).
ruff	Lint _G e formatt _G per il codice Backend Python (<code>ruff check</code> , <code>ruff format</code>).
mypy	Type checker statico per il Backend Python.
eslint, con oxlint	Lint _G per il codice Frontend TypeScript/Vue.js; oxlint esegue un primo passaggio più rapido sugli stessi file, integrato nella stessa pipeline.
prettier	Formatt _G per il codice Frontend (TypeScript, Vue, CSS).
vue-tsc	Type checker statico per componenti Vue.js e codice TypeScript del Frontend.
StarUML	Modellazione UML _G e produzione dei diagrammi architetturali.

Tutti gli strumenti di analisi statica e i framework di test (Sezione 2.5) sono richiamati da un comando unico, `make ci`, nell'ordine: analisi statica → type check → test → copertura, per Backend e Frontend in parallelo. Un'analisi statica fallita blocca l'esecuzione dei test corrispondenti, per un riscontro più rapido. Lo stesso comando viene eseguito automaticamente da GitHub Actions a ogni Pull Request.

2.7 Motivazioni delle scelte tecnologiche

Le tabelle precedenti riportano, tecnologia per tecnologia, la motivazione puntuale della scelta; questa sottosezione ne offre una sintesi trasversale. L'analisi comparativa completa delle alternative scartate per ciascuna tecnologia è riportata in [Analisi dello Stato dell'Arte](#) (Sezioni 2–3), a cui si rimanda per il dettaglio dei criteri di valutazione e dei pesi assegnati.

In sintesi, tre criteri hanno guidato trasversalmente le scelte tecnologiche del gruppo:

- **Coerenza con l'assenza di persistenza server-side** (Sezione 2.4): non essendo previsto alcun database nello scope obbligatorio (R4-V-Op non implementato), lo stack Backend non include un ORM né librerie di gestione dati strutturati oltre ai DTO di validazione (Pydantic); questo ha inoltre semplificato l'architettura di deployment a due soli container (Sezione 3.2).
- **Aderenza ai vincoli imposti dal capitolato**: la scelta di FastAPI e dello standard di comunicazione OpenAI-compatibile discende direttamente da R2-V-O e R3-V-O; la libertà di scelta del framework Frontend, non imposta dal capitolato (R11-V-O), ha invece lasciato margine di valutazione autonoma, esercitata a favore di Vue.js per la curva di apprendimento più contenuta rispetto alle alternative considerate.
- **Competenze pregresse del gruppo**: a parità di idoneità tecnica tra alternative comparabili (es. Python per il Backend), è stata data priorità alle tecnologie con cui alcuni membri del gruppo avevano già familiarità, per ridurre il rischio tecnico nelle fasi iniziali di sviluppo.

3 Architettura del Sistema

3.1 Architettura Generale

Il sistema *Second Brain* è basato su un'architettura di tipo Client-Server_G per garantire la separazione delle responsabilità (*Separation of Concerns*) tra la gestione dell'interfaccia utente_G (editor Markdown) e l'orchestrazione delle chiamate agli LLM.

Il sistema è composto da due soli componenti software: **Web Application** (Frontend) e **Backend API**, che comunicano tramite un'unica interfaccia REST (**IBackendREST**). Il Frontend non comunica mai direttamente con il servizio LLM: infatti, ogni richiesta di elaborazione AI transita dal Backend, che funge da unico punto di orchestrazione verso il provider esterno. Questa scelta, oltre a rispettare il vincolo R2-V-O sullo standard delle chiamate API, evita di esporre nel browser eventuali credenziali di accesso al servizio LLM.

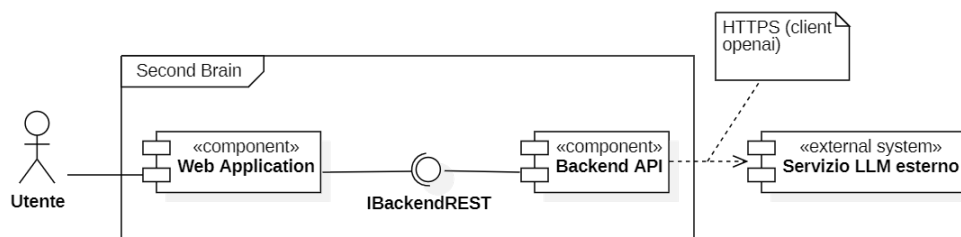


Figura 1: Diagramma dei componenti del sistema.

3.2 Architettura di Deployment

Frontend e Backend sono eseguiti in due container Docker distinti, orchestrati tramite Docker Compose:

- **frontend**: applicazione Vue.js, servita in sviluppo dal dev server Vite sulla porta 5173;
- **backend**: servizio FastAPI eseguito tramite Uvicorn sulla porta 8000, espone le API REST consumate dal frontend e contatta il servizio LLM esterno tramite HTTPS, con la relativa API key iniettata come variabile d'ambiente.

Questa scelta di deployment corrisponde a un'**architettura a due moduli deployabili indipendentemente (*two-monolith*)**, ciascuno containerizzato separatamente ma non ulteriormente suddiviso al proprio interno.

È stata preferita tale architettura piuttosto che:

- un **monolite unico indifferenziato**, perchè avrebbe accoppiato il deploy del Frontend (statico) a quello del Backend (con dipendenze verso servizi LLM esterni), impedendo aggiornamenti o riavvii indipendenti dei due componenti;
- una **scomposizione a microservizi** (es. un servizio per ciascuna operazione AI) perchè non sarebbe giustificata dalla scala del sistema, che espone un dominio applicativo singolo e un volume di richieste limitato, inoltre, introdurrebbe complessità di orchestrazione (service discovery, API gateway) senza benefici concreti, tanto più in assenza di persistenza (Sezione 2.4).

La granularità scelta consente comunque deploy e scalabilità indipendenti dei due componenti, preservando la possibilità di un'evoluzione più granulare qualora emergessero nuovi requisiti.

Il server di sviluppo Vite del container **frontend** inoltra le richieste verso `/api` al container **backend** attraverso la rete Docker interna; non è presente alcun container di persistenza, coerentemente con la scelta descritta in Sezione 2.4 di non implementare la persistenza remota (R4-V-Op, R78–R86).

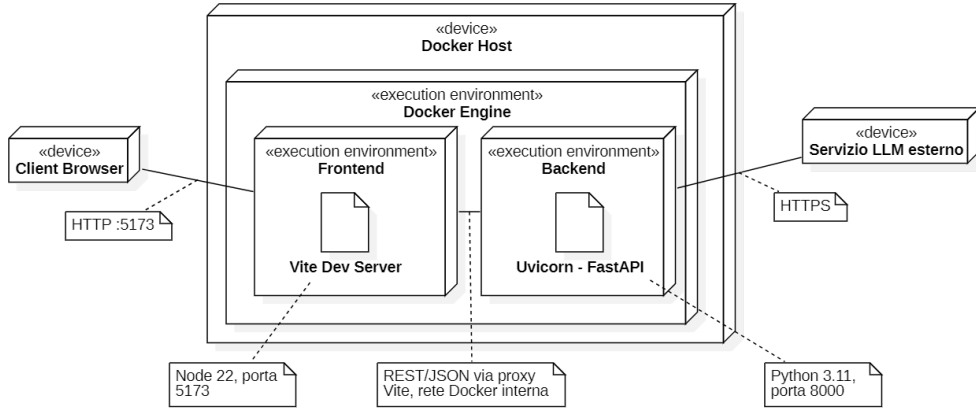


Figura 2: Diagramma UML di Deployment.

3.3 Architettura del Frontend

Il Frontend gestisce l'interazione dell'utente con l'editor Markdown e la visualizzazione dello stato delle operazioni AI. Il pattern architetturale adottato è **Model-View-Controller_G** nella versione *Pull Model_G*.

Questa separazione esplicita, indipendente dal framework di rendering, mantiene la logica di dominio (Model e Controller) testabile in isolamento: i componenti Vue.js costituiscono l'implementazione concreta della View, limitandosi a riflettere nel `templateG` lo stato esposto dal relativo oggetto View e a inoltrare gli eventi dell'utente, senza contenere essi stessi logica applicativa.

Vue.js è stato scelto consapevole che, a differenza di framework più strutturati, non fornisce nativamente un sistema di `Dependency InjectionG` né impone pattern architetturali quindi questi sono stati progettati e introdotti autonomamente dal gruppo (Sezione 6).

L'orchestrazione tra View e Model è affidata a due Controller distinti, coerenti con la separazione tra editing e interazioni AI: **EditorController** media tra **EditorView** e **NoteModel** (creazione ed esecuzione dei comandi di editing), mentre **AIController** media tra **AIPanelView** e **AIRequestModel** (avvio delle richieste AI e gestione delle proposte).

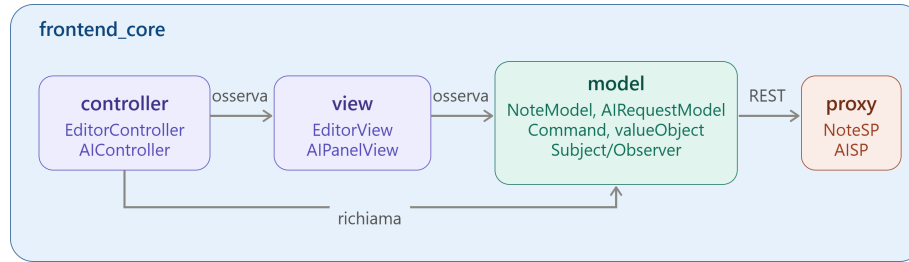
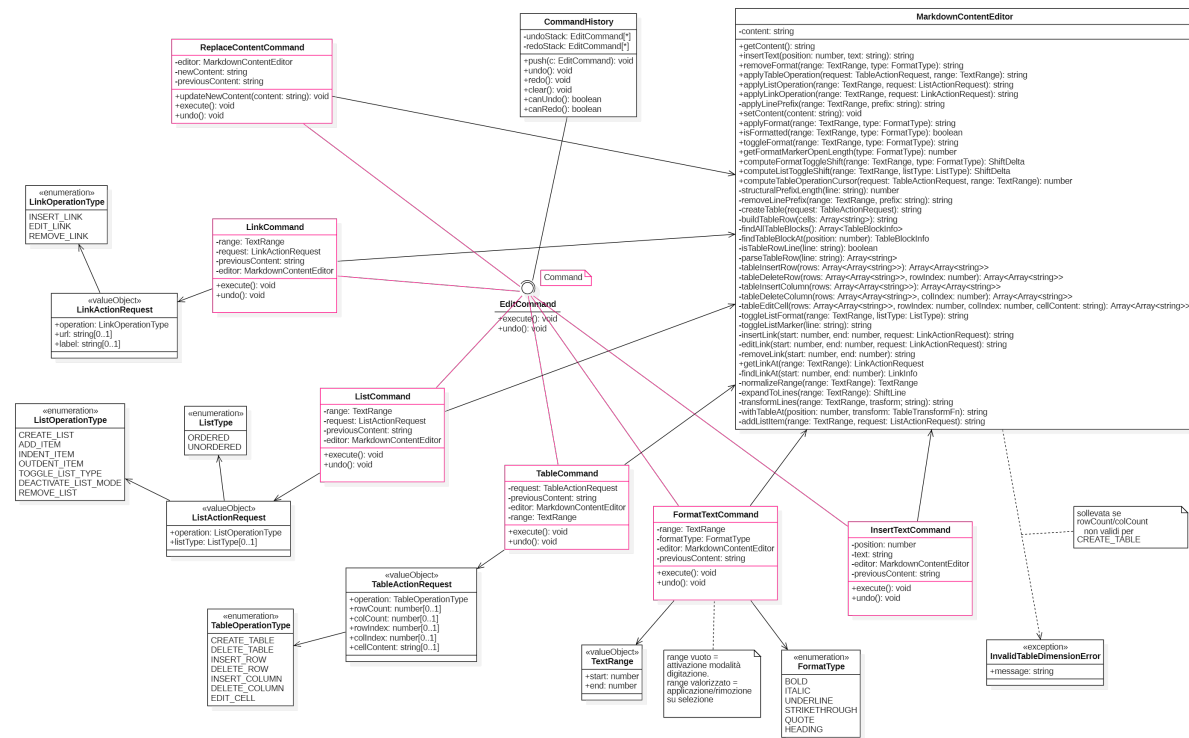


Figura 3: Diagramma dei package del Frontend.

3.3.1 Gestione dello Stato e dell'Editor

Il `NoteModel` mantiene lo stato osservabile della nota corrente (contenuto, identificativo, flag di modifica non salvata) e delega la manipolazione testuale a un `MarkdownContentEditor`. Ogni modifica (formattazione, tabelle, elenchi, link, inserimento di testo) è incapsulata in un comando (Sezione 6) eseguito tramite `NoteModel.executeCommand()`, che aggiorna una `CommandHistory` a due pile per supportare `UndoG`/`RedoG` (R43-F-O, R44-F-O) in modo uniforme per tutte le operazioni di editing.

Figura 4: Diagramma delle classi: `MarkdownContentEditor` e la famiglia di comandi di editing (pattern Command).

Il salvataggio e l'apertura della nota (R75-F-O, R76-F-O) sono delegati tramite l'interfaccia `NoteService` a un'implementazione basata sul File System locale, così da non vincolare il Model a un meccanismo di persistenza specifico; la scelta tra creazione di un nuovo file (`showSaveFilePicker`) e sovrascrittura di un file già aperto è gestita internamente al

`NoteServiceProxy`, che mantiene l'associazione tra identificativo di nota e handle di file ottenuto al primo salvataggio: `NoteModel` resta ignaro di questo dettaglio, coerentemente con l'incapsulamento garantito dal pattern `ProxyG` (Sezione 6).

La File System Access API (`showSaveFilePicker`, `showOpenFilePicker`) non è supportata nativamente da tutti i browser richiesti da R5-V-O: su Firefox e Safari, `NoteServiceProxy` rileva l'assenza dell'API a runtime e ricade su un fallback basato su API standard equivalenti: il download del file tramite `Blob` ed un elemento `<a download>` per il salvataggio, un elemento `<input type="file">` nascosto per l'apertura; mantenendo verso `NoteModel` lo stesso contratto `NoteService`, che quindi non deve conoscere questa differenza (R13-Q-D).

L'esportazione della nota (R77-F-O) segue lo stesso principio di indirezione: `NoteModel` espone `exportContent(format)`, che delega a un'istanza di `ExportService` (Proxy) iniettata opzionalmente nel costruttore, mantenendo coerente il pattern di Dependency Injection già adottato per `NoteService` e `AIService` invece di far dipendere `App.vue` direttamente dal Proxy di esportazione.

3.3.2 Gestione Interazioni AI (Widget/Pannello AI)

L'`AIRequestModel` mantiene lo stato della richiesta AI corrente come un tipo dato algebrico (union discriminata `TypeScript`) con quattro varianti: `IdleState`, `ProcessingState`, `ProposalReadyState` (con la `Proposal` generata) ed `ErrorState`. Abbiamo preferito un'unione discriminata `C` al pattern `State GoF`, che avrebbe introdotto una gerarchia di classi senza incapsulare alcun comportamento reale: le varianti (`IdleState`, `ProcessingState`, `ProposalReadyState`, `ErrorState`) sono semplici contenitori di dati privi di metodi polimorfici, mentre la logica di transizione resta centralizzata in `AIRequestModel`.

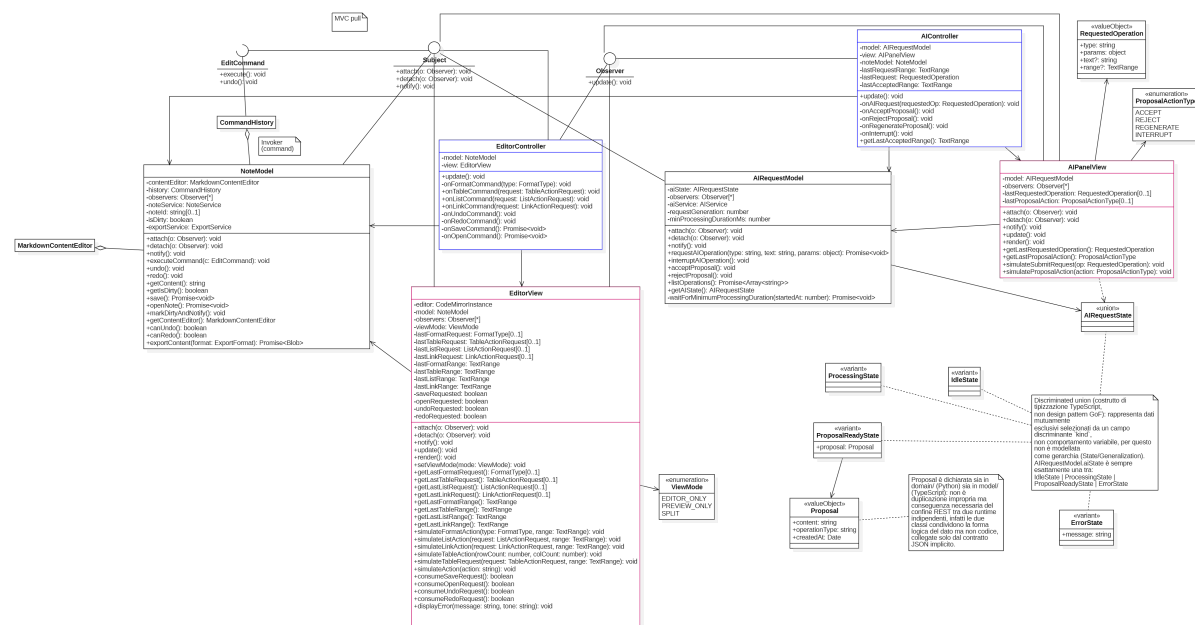


Figura 5: Diagramma delle classi: architettura MVC pull del Frontend (editing e interazioni AI) e rappresentazione dello stato di `AIRequestModel` come union discriminata.

Il ciclo di vita di una richiesta AI attraversa tre fasi distinte, ciascuna illustrata da un diagramma di sequenza dedicato in Sezione 6:

1. l'utente avvia la richiesta e riceve un riscontro visivo immediato (stato `Processing`, R72-F-O) con l'opzione di interruzione manuale (R73-F-O);
2. il Backend elabora la richiesta interagendo con l'LLM;
3. una volta risolta la Promise di `AIServiceProxy.requestOperation()`, aggiorna lo stato a `ProposalReadyState` e la View mostra la proposta con le azioni disponibili.

L'utente può quindi accettare, rifiutare o rigenerare la proposta (R69–R71): l'accettazione riusa lo stesso meccanismo di comando dell'editing manuale, inserendo il testo proposto tramite un comando eseguito su `NoteModel` (pattern `CommandG`).

Le richieste verso il Backend sono mediate dall'interfaccia `AIService`, la cui implementazione concreta (Proxy) incapsula i dettagli della chiamata REST. La Figura 6 riassume, insieme, le tre interfacce Proxy verso il Backend introdotte in questa sezione (`NoteService`, `ExportService`, `AIService`), con le rispettive implementazioni concrete.

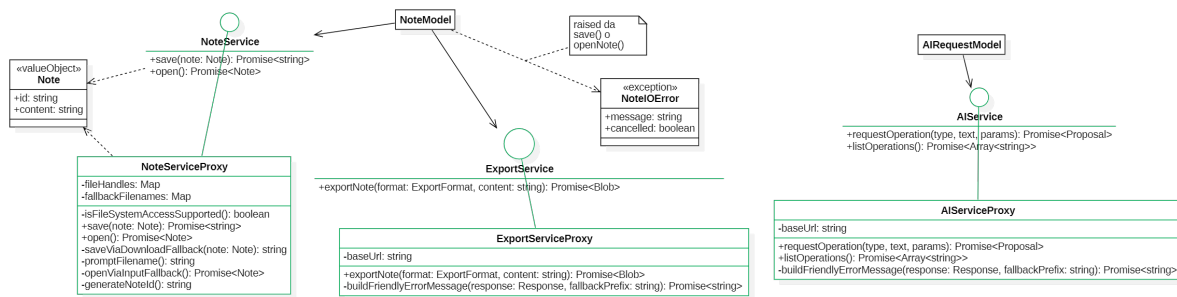


Figura 6: Diagramma delle classi: interfacce Proxy verso i servizi Backend (`NoteService`, `ExportService`, `AIService`) e relative implementazioni.

3.4 Architettura del Backend

Il Backend funge da orchestratore centrale e da strato di astrazione verso i provider LLM esterni. È organizzato secondo un'architettura a livelli, articolata in due livelli principali:

1. **Livello di Presentazione** (package `API.business_layer`): router REST, DTO, Dependency Injection;
2. **Livello di Business Logic** (package `AI.Domain`): logica di costruzione dei prompt e di integrazione con il servizio LLM esterno, indipendente dal framework web.

A questi si affianca il package `Export`, che non fa parte della catena di elaborazione delle richieste AI, bensì implementa in modo isolato e autosufficiente la conversione della nota nei formati richiesti, riuscendo lo stesso principio di separazione delle responsabilità senza costituire un ulteriore livello architetturale.

La **scomposizione a livelli** riflette la scala contenuta del sistema garantendo una separazione sufficiente a mantenere il dominio AI indipendente sia dal framework HTTP che dal provider LLM concreto.

La scelta tra un'architettura Esagonale_G e una a Layer è stata oggetto di un confronto esplicito con il referente aziendale (Verbale del 30/07/2026, decisione VE-8.1): sono stati messi a confronto un'architettura Esagonale, più pulita e flessibile rispetto a possibili evoluzioni future ma

a fronte di una maggiore complessità realizzativa, e un'architettura a Layer_G, più naturale per applicazioni di questo tipo e meno onerosa da realizzare quando non sono previste evoluzioni architetturali significative. Non essendo previste, per *Second Brain*, estensioni quali la persistenza remota o l'integrazione con provider LLM multipli all'interno della stessa richiesta (R4-V-Op, R6-V-O), è stata adottata l'architettura a Layer, riservando comunque, tramite la separazione tra Presentazione e Business Logic già descritta, un margine di evoluzione verso un'architettura più disaccoppiata qualora emergessero nuovi requisiti.

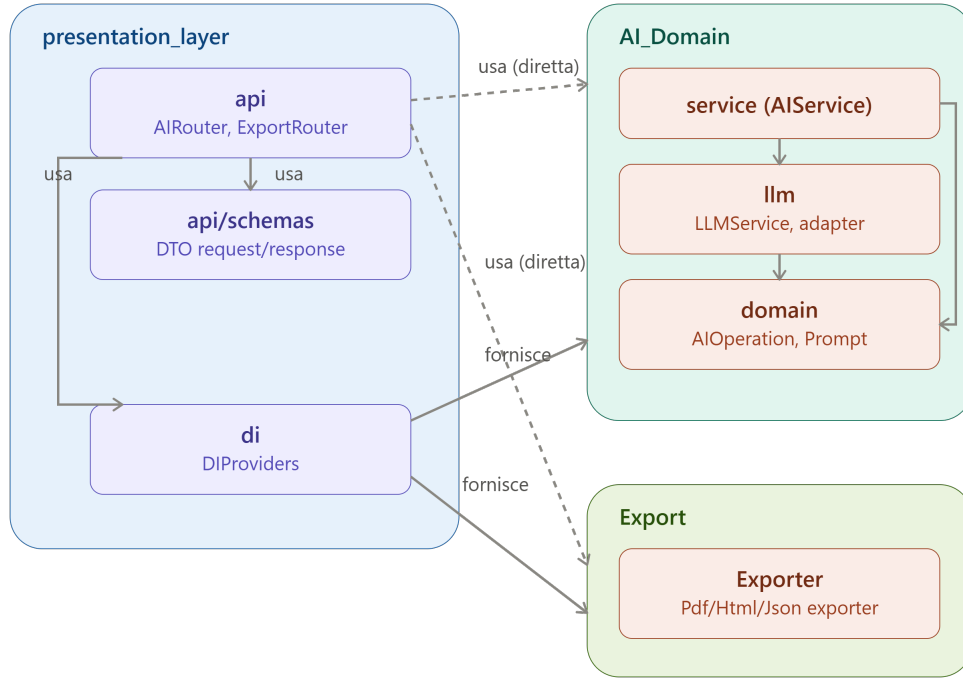


Figura 7: Diagramma dei package del Backend.

3.4.1 Livello di Presentazione

I router FastAPI (`AIRouter`, `ExportRouter`¹) ricevono le richieste dal client, le validano tramite i DTO di `api/schemas` (`AIOperationRequest`, `ExportRequest`) e delegano l'elaborazione ai servizi applicativi, ottenuti tramite Dependency Injection (`DProviders`). Un gestore di errori centralizzato (`ErrorHandlers`) traduce le eccezioni di dominio (`AIDomainError`, `ExportError`) in risposte HTTP coerenti (Sezione 5.2). I router non contengono logica di dominio: traducono richieste HTTP in chiamate al livello di Business Logic e le risposte in schemi Pydantic.

¹Nomi informali usati in questo documento per riferirsi rispettivamente al router definito in `api/ai_router.py` e a quello in `api/export_router.py`: nel codice sono variabili modulo (`router = APIRouter(...)`), non classi. Allo stesso modo, `DProviders` indica le funzioni fabbrica del modulo `di/providers.py` (`get_ai_service()`, `get_exporter(format)`), ed `ErrorHandlers` le funzioni di gestione errori del modulo `api/error_handlers.py` (`handle_ai_domain_error()`, `handle_export_error()`).

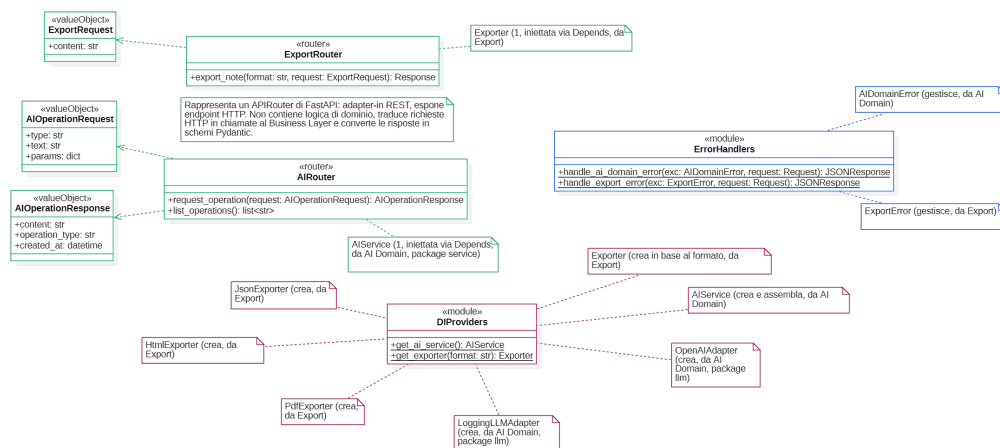


Figura 8: Diagramma delle classi del livello di Presentazione e DI del Backend.

3.4.2 Livello di Business Logic (Core LLM)

L'**AIService** è il motore centrale del dominio AI: riceve il tipo di operazione richiesta (riassunto, traduzione, riscrittura, Distant Writing, o una delle sei prospettive dei “Cappelli di De Bono”), la risolve tramite un'**AIOperationFactory** auto-registrante, e ne invoca `build_prompt()` in modo polimorfo. Ogni operazione costruisce il proprio **Prompt** tramite un **PromptBuilder** fluente, isolando la logica specifica di ciascuna trasformazione testuale (Sezione 6, pattern Strategy_G, Builder_G e Simple Factory_G).

Il pacchetto `AI_Domain/llm`, incluso nello stesso diagramma, maschera le specificità del provider LLM concreto dietro l'interfaccia `LLMService`: un decorator concreto (`LoggingLLMAdapter`) aggiunge la funzionalità trasversale di logging senza modificare l'implementazione di base; la classe astratta `LLMServiceDecorator` da cui deriva è pensata per accogliere eventuali ulteriori decorator (es. un futuro livello di caching) componibili nella stessa catena, senza impatti su `AIService`. L'ultimo anello della catena è l'`OpenAIAdapter`, verso l'API compatibile con lo standard OpenAI (R2-V-O, R3-V-O).

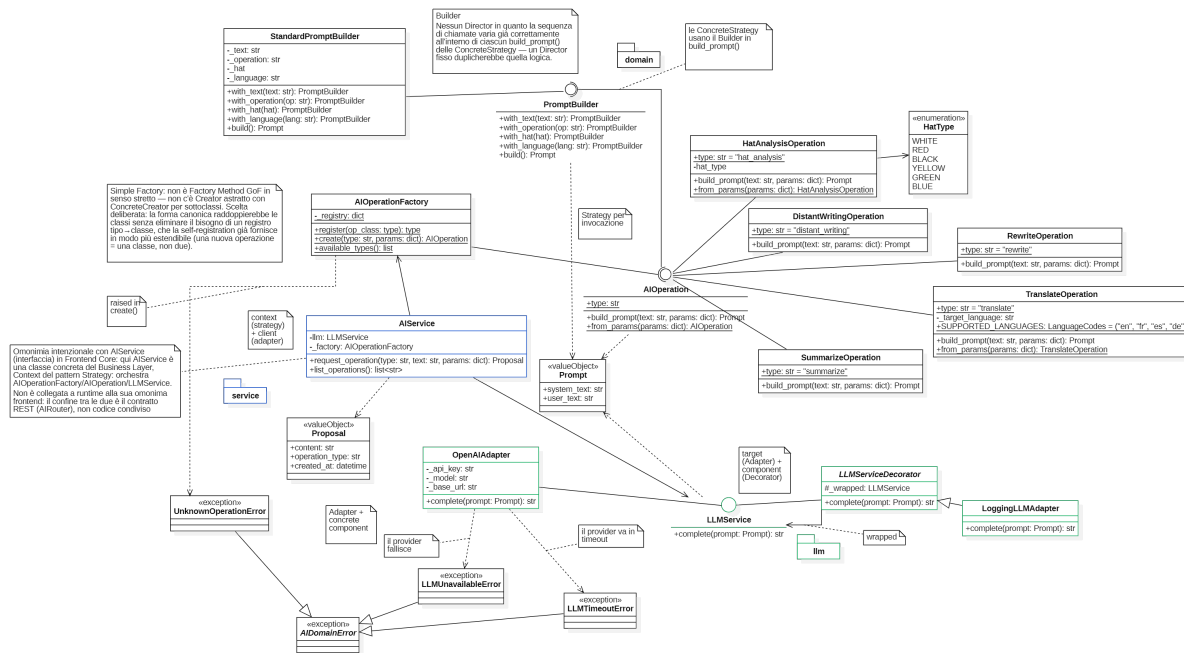


Figura 9: Diagramma delle classi del dominio AI: operazioni, costruzione dei prompt e integrazione con il servizio LLM.

3.4.3 Componente di Esportazione (Export)

La conversione della nota nei formati di esportazione richiesti (PDF, HTML, JSON)(R77-F-O) è isolata nel package **Export**, strutturato attorno al pattern Template Method_G (Sezione 6): la classe astratta **Exporter** fissa i passi comuni dell'esportazione (`export()`, `_prepare_content()`) mentre delega alle sottoclassi (**PdfExporter**, **HtmlExporter**, **JsonExporter**) solo la conversione nel formato finale (`_convert_format()`). `DIProviders.get_exporter(format)` risolve l'esportatore concreto in base al formato richiesto da **ExportRouter**, senza che **Export** debba conoscere né il livello di Presentazione né quello di Business Logic.

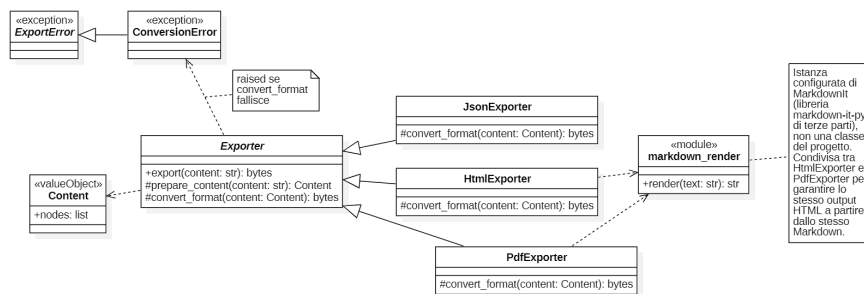


Figura 10: Diagramma delle classi del package Export (Template Method).

4 Decomposizione del Sistema

Questa sezione raccoglie in forma tabellare le responsabilità, le interfacce esposte e le dipendenze dei componenti architetturali principali già introdotti in Sezione 3.3 e Sezione 3.4, ai fini di una consultazione rapida e uniforme. Non sostituisce i diagrammi delle classi (Figure 4, 5, 6, 8, 9, 10), a cui si rimanda per il dettaglio di attributi e metodi.

Tabella 6: Componenti Architetturali di Alto Livello

Componente	Responsabilità	Interfacce	Dipendenze
NoteModel	Mantiene lo stato osservabile della nota corrente ed esegue i comandi di editing tramite <code>executeCommand()</code> ; delega salvataggio/apertura a <code>NoteService</code> ed esportazione (<code>exportContent()</code>) a <code>ExportService</code> , se iniettato.	Realizza <code>Subject</code> ; espone <code>executeCommand()</code> , <code>exportContent()</code> .	<code>MarkdownContentEditor</code> , <code>CommandHistory</code> , <code>NoteService</code> , <code>ExportService</code>
MarkdownContentEditor	Manipola direttamente il contenuto testuale Markdown della nota (inserimento, sostituzione, formattazione, validazione delle dimensioni di una tabella); non conosce comandi, <code>undo/redo</code> né persistenza.	Espone i metodi di manipolazione testuale invocati dai singoli <code>EditCommand</code> in fase di <code>execute()/undo()</code> .	Nessuna dipendenza esterna al Frontend
AIRequestModel	Mantiene lo stato della richiesta AI corrente come unione discriminata _G (<code>Idle/Processing/ProposalReady/Error</code>).	Realizza <code>Subject</code> .	<code>AIService</code> (Proxy, Frontend)
EditorView / AIPanelView	Riflettono nel template lo stato pullato dal Model; inoltrano gli eventi utente al Controller.	Realizzano <code>Subject</code> e <code>Observer</code> .	<code>NoteModel</code> / <code>AIRequestModel</code>
EditorController / AIController	Orchestrano la creazione dei comandi di editing e l'avvio delle richieste AI a partire dagli eventi della View.	Realizzano <code>Observer</code> .	<code>EditorView/NoteModel</code> , <code>AIPanelView/AIRequestModel</code>
CommandHistory, famiglia <code>EditCommand</code>	Eseguono e archiviano i comandi di editing in due pile, per supportare <code>Undo/Redo</code> uniforme su tutte le operazioni.	<code>execute()</code> , <code>undo()</code> per ogni <code>EditCommand</code> .	Nessuna dipendenza esterna al Frontend
Continua nella pagina successiva...			

Tabella 6 – Continua dalla pagina precedente

Componente	Responsabilità	Interfacce	Dipendenze
NoteServiceProxy / AIServiceProxy / ExportServiceProxy	Incapsulano rispettivamente l'accesso al file system locale (<code>showSaveFilePicker/showOpenFilePicker</code> , con fallback via download/<input type=file> per Firefox/Safari), la comunicazione REST col Backend per le operazioni AI e per l'esportazione.	Implementano <code>NoteService / AIService / ExportService</code> .	File System Access API del browser (o fallback) / IBackendREST
AIRouter, ExportRouter	Ricevono le richieste HTTP, le validano tramite i DTO e delegano l'elaborazione al livello di Business Logic.	Espongono IBackendREST.	DIPProviders, DTO di api/schemas
AIService (Backend)	Risolve l'operazione richiesta tramite Simple Factory _G e ne invoca <code>build_prompt()</code> in modo polimorfico.	<code>request_operation()</code> , <code>list_operations()</code> .	AIOperationFactory, LLMService
AIOperationFactory, famiglia AIOperation	Auto-registrano e istanziano l'operazione AI corretta (riassunto, traduzione, riscrittura, Distant Writing, Sei Cappelli).	<code>register()</code> , <code>create()</code> ; ogni AIOperation implementa <code>build_prompt()</code> .	StandardPromptBuilder
LLMService, decorator LoggingLLMAdapter, OpenAIAdapter	Mascherano le specificità del provider LLM concreto dietro un'interfaccia comune; il decorator aggiunge logging senza modificarne l'implementazione.	<code>complete(prompt)</code> .	API esterna compatibile OpenAI
Exporter, famiglia (PdfExporter, HtmlExporter, JsonExporter)	Convertono il contenuto Markdown della nota nel formato di esportazione richiesto, secondo lo scheletro fisso del Template Method _G .	<code>export()</code> ; ogni sottoclasse implementa <code>_convert_format()</code> .	Nessuna dipendenza dal livello di Presentazione o Business Logic
ErrorHandler, gerarchia AI DomainError	Intercettano le eccezioni di dominio e le traducono in risposte HTTP normalizzate.	<code>handle_ai_domain_error()</code> , <code>handle_export_error()</code> .	AIRouter, ExportRouter
DIPProviders	Espongono le funzioni fabbrica per ottenere le collaborazioni concrete (servizio AI, esportatore per formato) a runtime.	<code>get_ai_service()</code> , <code>get_exporter(format)</code> .	Settings, Output Adapter concreti

5 Interfaccia API e Comunicazione

Al fine di garantire il disaccoppiamento tra l'editor Markdown (Frontend) e l'elaboratore di prompt (Backend), la comunicazione avviene tramite API **REST** su JSON, esposta dal Backend FastAPI e consumata dal Frontend tramite il pattern Proxy (Sezione 6).

5.1 Struttura dei Payload AI

La comunicazione relativa alle operazioni di Intelligenza Artificiale (riassunto, traduzione, riscrittura, Sei Cappelli, Distant Writing) è normalizzata su un'unica struttura di richiesta e di risposta, validata da FastAPI tramite modelli tipizzati (`api/schemas`), indipendentemente dall'operazione richiesta.

DTO	Campo	Descrizione
AIOperationRequest	type	Tipo di operazione richiesta (es. <code>summarize</code> , <code>translate</code> , <code>rewrite</code> , <code>distant_writing</code> , <code>hat_analysis</code>), risolto dinamicamente da <code>AIOperationFactory</code> . Le sei prospettive dei Cappelli di De Bono condividono un unico <code>type</code> (<code>hat_analysis</code>): il cappello specifico è indicato nel campo <code>params.hat_type</code> (es. <code>white</code> , <code>red</code>), non nel <code>type</code> stesso.
	text	Testo selezionato dall'utente su cui operare (contesto).
	params	Parametri aggiuntivi specifici dell'operazione (es. <code>target_language</code> per la traduzione, <code>hat_type</code> per l'analisi critica, <code>user_prompt</code> per il Distant Writing).
AIOperationResponse	content	Testo generato dall'LLM, restituito come proposta.
	operation_type	Tipo di operazione a cui la risposta si riferisce, per consentire al Frontend di associare correttamente la proposta alla richiesta.
	created_at	Timestamp di generazione, utile per l'ordinamento e l'eventuale invalidazione della proposta.
ExportRequest	content	Contenuto Markdown della nota da convertire nel formato di esportazione richiesto (PDF, HTML o JSON).

Questa struttura unificata consente di aggiungere nuove operazioni AI (rispettando l'Open Closed Principle_G, Sezione 6) senza introdurre nuovi endpoint o nuovi contratti di comunicazione in quanto è sufficiente registrare una nuova strategia lato Backend.

5.2 Gestione degli Errori e Fallback_G

Le anomalie sono modellate come una gerarchia di eccezioni di dominio (**AIDomainError**), distinta dalle eccezioni tecniche di rete o di parsing:

- **LLMUnavailableError**: il servizio LLM esterno non è raggiungibile (assenza di connettività o servizio non configurato);
- **LLMTimeoutError**: la richiesta al modello LLM eccede il timeout configurato;
- **UnknownOperationError**: il tipo di operazione richiesto non è registrato in **AIOperationFactory**.

Un gestore di eccezioni centralizzato (**ErrorHandlers**) intercetta queste eccezioni all'interno dei router e le traduce in risposte HTTP normalizzate (codice di stato e messaggio), che il Frontend interpreta per notificare l'utente in caso di errore durante una qualsiasi delle operazioni AI (R49-F-O, R51-F-O, R54-F-O, R56-F-O, R58-F-O, R60-F-O, R62-F-O, R64-F-O, R66-F-O, R68-F-O), mantenendo il documento invariato e permettendo il rifiuto o la rigenerazione della proposta (R70-F-O, R71-F-O).

Coerentemente con il vincolo R6-V-O, ogni richiesta di elaborazione AI è indirizzata a un singolo modello LLM per l'intera durata della richiesta: il Backend non concatena chiamate a modelli differenti all'interno della stessa operazione.

6 Design Pattern Utilizzati

Di seguito sono illustrati i design pattern adottati per risolvere problematiche ricorrenti e garantire la scalabilità del codice di *Second Brain*. Ogni pattern è stato introdotto solo laddove i diagrammi architetturali ne mostrano un utilizzo concreto.

6.1 Pattern Creazionali

6.1.1 Simple Factory

Problema risolto	AIService deve creare l'istanza dell'operazione AI corretta a partire da una stringa (il tipo di operazione richiesto dal Frontend) senza contenere un'istruzione condizionale per ogni tipo esistente, e nuove operazioni devono potersi aggiungere rispettando l'Open Closed Principle.
Soluzione adottata	AIOperationFactory mantiene un registro (<code>_registry</code>) e offre <code>register(op_class)</code> per l'auto-registrazione delle classi operazione e <code>create(type, params)</code> per istanziarle; solleva <code>UnknownOperationError</code> se il tipo non è registrato. Non è un Factory Method GoF in senso stretto: infatti, non esiste un Creator astratto con ConcreteCreator per sottoclasse e tale scelta dovuta dal fatto che la forma canonica del Factory Method raddoppierebbe le classi (una fabbrica per ogni operazione) senza eliminare il bisogno di un registro tipo→classe, che la self-registration già fornisce in modo più estendibile.
Utilizzo nel progetto	<code>AIService.request_operation</code> delega alla factory il compito di istanziare l'operazione corretta a partire dal campo <code>type</code> ricevuto dall'API, per ciascuna delle operazioni AI previste (riassunto, traduzione, riscrittura, Distant Writing, sei Cappelli).
Vantaggi	L'aggiunta di una nuova operazione AI richiede solo la creazione di una nuova classe e la sua auto-registrazione, senza modificare <code>AIService</code> né introdurre nuovi condizionali (Open/Closed Principle); il registro rende inoltre immediato esporre a runtime l'elenco delle operazioni disponibili tramite <code>available_types()</code> .

6.1.2 Builder

Problema risolto	La costruzione di un <code>Prompt</code> richiede di comporre più parti opzionali (testo, tipo di operazione, cappello selezionato per l'analisi critica, lingua di destinazione per la traduzione) in modo incrementale, evitando costruttori con molti parametri opzionali.
-------------------------	---

Soluzione adottata	L'interfaccia <code>PromptBuilder</code> espone metodi fluenti (<code>with_text</code> , <code>with_operation</code> , <code>with_hat</code> , <code>with_language</code>) che restituiscono il builder stesso, e un metodo <code>build()</code> che produce il <code>Prompt</code> finale. <code>StandardPromptBuilder</code> è l'unica implementazione concreta, usata sia come Builder che come Context della Strategy (Sezione 5.3.1): ogni <code>ConcreteStrategy</code> (operazione AI) lo invoca direttamente in <code>build_prompt()</code> . Non è previsto un Director in quanto la sequenza di chiamate al builder varia già correttamente all'interno di ciascun <code>build_prompt()</code> delle diverse operazioni, e un Director fisso duplicherebbe quella logica invece di semplificarla.
Utilizzo nel progetto	Ogni operazione AI usa il builder per assemblare il prompt in base ai soli parametri rilevanti: l'operazione di riassunto usa <code>with_text/with_operation/build</code> , mentre l'analisi secondo i Sei Cappelli aggiunge <code>with_hat</code> .
Vantaggi	Separa la costruzione incrementale del <code>Prompt</code> dalla sua rappresentazione finale, permettendo a ciascuna operazione di comporre solo i parametri per essa rilevanti, senza costruttori sovraccarichi o parametri opzionali non pertinenti.

6.2 Pattern Strutturali

6.2.1 Decorator

Problema risolto	Al servizio di completamento LLM va aggiunta, in modo componibile e senza modificarne l'implementazione, la funzionalità trasversale di logging delle richieste/risposte, mantenendo la possibilità di comporre in futuro ulteriori funzionalità analoghe (es. caching) senza toccare <code>AIService</code> .
Soluzione adottata	<code>LLMServiceDecorator</code> è una classe astratta che realizza <code>LLMService</code> e mantiene un riferimento a un'istanza <code>wrapped: LLMService</code> ; il decorator concreto attualmente implementato (<code>LoggingLLMAdapter</code>) ne arricchisce il comportamento prima/dopo aver delegato la chiamata.
Utilizzo nel progetto	La catena effettiva, visibile nel diagramma di sequenza "Richiesta AI (tratto REST→LLM)" (Figura 12), è <code>LoggingLLMAdapter</code> → <code>OpenAIAdapter</code> (il target concreto), componibile e configurabile all'avvio dell'applicazione senza toccare <code>AIService</code> .
Vantaggi	Ulteriori funzionalità trasversali (es. un futuro livello di caching) potrebbero essere aggiunte componendo la catena in fase di configurazione _G , senza modificare <code>OpenAIAdapter</code> né <code>AIService</code> , rispettando il Single Responsibility Principle per ciascun anello.

6.2.2 Adapter_G

Problema risolto	<code>AIService</code> deve rimanere indipendente dal provider LLM concreto; l'interfaccia esposta dal provider esterno (API compatibile con lo standard OpenAI) non coincide con l'interfaccia <code>LLMService</code> usata internamente.
Soluzione adottata	<code>OpenAIAdapter</code> realizza <code>LLMService</code> e incapsula i dettagli di connessione al provider (<code>api_key</code> , <code>model</code> , <code>base_url</code>), traducendo <code>complete(prompt)</code> in una richiesta HTTP verso il servizio LLM e gestendo le eccezioni specifiche (<code>LLMUnavailableError</code> se il provider fallisce, <code>LLMTimeoutError</code> se va in timeout).
Utilizzo nel progetto	Grazie a <code>base_url</code> configurabile, l'adapter permette di puntare indifferentemente ai modelli Gemma, ChatGPT-OSS o Qwen messi a disposizione dall'azienda proponente, senza modifiche al resto del sistema. L'infrastruttura attuale prevede l'utilizzo iniziale del modello <code>gemma4:12b</code> , con possibile ampliamento dei permessi di accesso al modello <code>Qwen3.6-35B-A3B</code> qualora necessario (R3-V-O): il passaggio tra i due richiede solo l'aggiornamento della configurazione dell'adapter, senza impatti sul resto del sistema. È al tempo stesso il concrete component su cui si applica la catena di Decorator descritta sopra.
Vantaggi	Isola <code>AIService</code> dai dettagli di uno specifico provider LLM, rendendo possibile sostituire modello o provider (Gemma, ChatGPT-OSS, Qwen) agendo solo sulla configurazione, senza impatti sul resto del dominio applicativo.

6.2.3 Proxy (remoto)

Problema risolto	Il Frontend deve invocare le funzionalità di note-taking e AI esposte dal Backend come se fossero locali, senza spargere nel codice applicativo i dettagli di trasporto (HTTP, base URL, serializzazione).
Soluzione adottata	Le interfacce <code>NoteService</code> , <code>AIService</code> ed <code>ExportService</code> (lato Frontend) definiscono le operazioni disponibili; le rispettive implementazioni Proxy le realizzano incapsulando la comunicazione con, rispettivamente, il file system locale (o il suo fallback) e il Backend REST.
Utilizzo nel progetto	<code>NoteModel</code> dipende solo dalle interfacce <code>NoteService</code> ed <code>ExportService</code> , <code>AIRquestModel</code> solo dall'interfaccia <code>AIService</code> : nessuna delle classi di Model conosce se l'implementazione sia locale o remota, facilitando anche il testing tramite implementazioni fittizie. È importante notare che si tratta di un'omonimia intenzionale con la classe concreta <code>AIService</code> del Backend (Sezione 3.4): le due entità vivono a runtime in linguaggi indipendenti e non condividono alcun codice, collegate solo dal contratto REST esposto da <code>AIRouter</code> .

Vantaggi	Il Model rimane completamente ignaro se l'operazione richiesta sia eseguita in locale o tramite rete, semplificando il testing (sostituzione con implementazioni fittizie) e isolando ogni futura evoluzione del trasporto dal resto del Frontend.
-----------------	--

6.2.4 Template Method

Problema risolto	L'esportazione di una nota segue sempre gli stessi passi generali (preparazione del contenuto, conversione nel formato di destinazione) ma il dettaglio della conversione dipende dal formato richiesto (PDF, HTML, JSON).
Soluzione adottata	La classe astratta <code>Exporter</code> definisce il metodo <code>template export(content: str): bytes</code> , che fissa la sequenza <code>_prepare_content() → _convert_format()</code> ; solo <code>_convert_format()</code> è il passo variabile, ridefinito da ciascuna sottoclasse (<code>PdfExporter</code> , <code>HtmlExporter</code> , <code>JsonExporter</code>). Un fallimento in fase di conversione solleva <code>ConversionError</code> (sottotipo di <code>ExportError</code>).
Utilizzo nel progetto	<code>ExportRouter.export_note(format, request)</code> risolve tramite <code>DIPProviders.get_exporter(format)</code> l'esportatore concreto e ne invoca <code>export()</code> , senza conoscere il formato specifico: aggiungere un nuovo formato richiede solo una nuova sottoclasse di <code>Exporter</code> .
Vantaggi	Garantisce che ogni formato di esportazione rispetti la stessa sequenza di passi, evitando duplicazione di codice tra gli <code>Exporter</code> concreti; l'aggiunta di un nuovo formato richiede solo l'implementazione di <code>_convert_format()</code> , senza toccare la logica comune.

6.3 Pattern Comportamentali

6.3.1 Strategy

Problema risolto	Le operazioni AI condividono la stessa interfaccia di utilizzo (costruzione di un <code>Prompt</code> a partire da un testo) ma differiscono nell'algoritmo di costruzione del prompt stesso; nuove operazioni devono potersi aggiungere senza modificare <code>AIService</code> .
Soluzione adottata	L'interfaccia <code>AIOperation</code> definisce <code>build_prompt(text, params)</code> ; ciascuna operazione concreta (riassunto, traduzione, riscrittura, Distant Writing, le sei varianti dell'analisi critica) la implementa con la propria logica, usando <code>StandardPromptBuilder</code> come contesto di costruzione.

Utilizzo nel progetto	<code>AIService</code> non conosce le operazioni concrete: riceve un'istanza dalla Simple Factory e la invoca in modo polimorfo, come illustrato nel diagramma di sequenza “Richiesta AI (tratto REST→LLM) (Figura 12)”.
Vantaggi	Permette di aggiungere nuove operazioni AI, incluse eventuali varianti dei Sei Cappelli, come nuove classi che implementano <code>AIOperation</code> , senza modificare <code>AIService</code> (Open/Closed Principle).

6.3.2 Command

Problema risolto	Le modifiche al testo della nota (formattazione, tabelle, elenchi, link, inserimento, sostituzione) devono poter essere annullate e ripetute in modo uniforme (R43-F-O, R44-F-O), senza che <code>NoteModel</code> conosca i dettagli di ciascuna operazione di editing.
Soluzione adottata	Un'interfaccia <code>EditCommand</code> definisce <code>execute()/undo()</code> ; ogni tipo di modifica la implementa incapsulando lo stato necessario per l'annullamento. Oltre ai comandi specifici per formattazione, tabelle, elenchi e link, <code>ReplaceContentCommand</code> è un comando generico che sostituisce l'intero contenuto con uno nuovo, mantenendo lo snapshot precedente per l'undo. Una <code>CommandHistory</code> mantiene due pile (undo/redo).
Utilizzo nel progetto	Il metodo <code>executeCommand(c)</code> di <code>NoteModel</code> delega a <code>CommandHistory</code> , che esegue e archivia il comando. <code>ReplaceContentCommand</code> è riusato in tre contesti distinti: per raggruppare in un'unica voce di history le sequenze di digitazione libera, per Taglia/Incolla, e per l'accettazione di una proposta AI, dove sostituisce il testo originariamente selezionato con il contenuto generato (diagramma di sequenza “Accetta proposta AI”) (Figura 14); solo nel caso limite in cui il range della selezione originaria non sia più disponibile (rete di sicurezza), l'accettazione ricade su <code>InsertTextCommand</code> , che inserisce il testo in coda al documento invece di sostituirlo.
Vantaggi	Uniforma undo/redo per qualsiasi tipo di modifica testuale, manuale o generata dall'AI, attraverso un'unica interfaccia, evitando di duplicare la logica di annullamento tra editing manuale, appunti e accettazione delle proposte AI.

6.3.3 Observer_G

Problema risolto	Model e View devono notificare i propri cambiamenti a più osservatori (View, Controller) senza conoscerne il tipo concreto, per mantenere il disaccoppiamento richiesto dal pattern MVC (Sezione 3.3).
-------------------------	--

Soluzione adottata	Un'interfaccia <code>Subject</code> definisce <code>attach/detach/notify</code> ; un'interfaccia <code>Observer</code> definisce <code>update()</code> . Il <code>Model</code> realizza <code>Subject</code> ; la <code>View</code> realizza sia <code>Subject</code> sia <code>Observer</code> ; il <code>Controller</code> realizza <code>Observer</code> .
Utilizzo nel progetto	Ogni modifica al contenuto della nota o allo stato di una richiesta AI propaga automaticamente l'aggiornamento a <code>View</code> e <code>Controller</code> , garantendo la visualizzazione continua dello stato richiesta da R72-F-O.
Vantaggi	Disaccoppia il <code>Model</code> dai suoi osservatori: nuovi componenti potrebbero sottoscrivere agli aggiornamenti di stato senza richiedere modifiche a <code>NoteModel</code> o <code>AIRequestModel</code> .

6.3.4 Dependency Injection

Problema risolto	Sia il Backend che il Frontend devono ottenere le proprie dipendenze concrete (rispettivamente: <code>AIService</code> ed esportatori per formato lato Backend; <code>NoteService</code> , <code>AIService</code> ed <code>ExportService</code> lato Frontend) senza costruirle direttamente all'interno delle classi che le usano, per restare testabili e disaccoppiati dall'implementazione concreta.
Soluzione adottata	Sul Backend, <code>DIProviders</code> espone funzioni fabbrica (<code>get_ai_service</code> , <code>get_exporter</code>) iniettate nei router dal meccanismo di Dependency Injection nativo di FastAPI (<code>Depends</code>), che le richiama automaticamente a ogni richiesta. Vue.js non fornisce nativamente un contenitore di DI equivalente: sul Frontend l'iniezione è quindi manuale, tramite costruttore. <code>App.vue</code> funge da <i>composition root</i> : è l'unico punto dell'applicazione in cui le implementazioni Proxy concrete (<code>NoteServiceProxy</code> , <code>AIServiceProxy</code> , <code>ExportServiceProxy</code>) vengono istanziate e passate ai costruttori di <code>NoteModel</code> ed <code>AIRequestModel</code> , che le ricevono già assemblate tramite le rispettive interfacce.
Utilizzo nel progetto	<code>AIRouter</code> ed <code>ExportRouter</code> dipendono da <code>DIProviders</code> per ottenere le proprie collaborazioni a runtime, invece di istanziarle staticamente. Sul Frontend, <code>App.vue</code> costruisce il grafo di dipendenze all'avvio dell'applicazione (<code>new NoteModel(editor, history, noteService, exportService)</code> , <code>new AIRequestModel(aiService)</code>); nessun'altra classe del Frontend istanzia mai direttamente un Proxy concreto.

Vantaggi

Rende i router e i Model testabili in isolamento iniettando implementazioni fittizie (di `AIService`, degli esportatori, o dei Proxy Frontend), e permette di sostituire la configurazione concreta (es. provider LLM lato Backend) senza modificare il codice dei consumatori. Sul Frontend, concentrare l'istanziatura in un unico composition root (`App.vue`) rende esplicito e verificabile a colpo d'occhio l'intero grafo delle dipendenze dell'applicazione, pur in assenza di un framework di DI dedicato.

7 Flusso dei Dati (Data Flow)

Questa sezione illustra il ciclo di vita delle informazioni all'interno del sistema, partendo dall'interazione dell'utente fino all'ottenimento del risultato generato dall'AI o alla persistenza della nota.

Le frecce continue con punta piena rappresentano chiamate sincrone, le frecce tratteggiate le risposte (reply), le frecce continue con punta stilizzata rappresentano chiamate asincrone, infine, quelle contrassegnate «create» istanziano un nuovo oggetto comando. I riquadri tratteggiati etichettati **alt** rappresentano rami alternativi mutuamente esclusivi, ciascuno introdotto da una guardia tra parentesi quadre (es. `[range definito]`): solo il ramo la cui condizione è verificata viene eseguito.

7.1 Elaborazione di un'operazione AI

Il flusso, dalla richiesta dell'utente alla visualizzazione della proposta, attraversa tre diagrammi di sequenza in successione, ciascuno con una responsabilità netta: il primo e il terzo rimangono all'interno del Frontend (rispettivamente l'avvio della richiesta e la ricezione della risposta), mentre il secondo descrive l'intero tratto di rete dal Frontend verso il Backend e l'LLM esterno.

1a – Richiesta operazione AI (tratto Frontend). Il flusso ha inizio quando l'Utente seleziona una porzione di testo all'interno dell'editor e richiede un'operazione specifica (ad esempio, cliccando su "Riassumi"). Questa interazione viene catturata direttamente dal componente di View `AIPanelView`, che aggiorna il proprio stato registrando la richiesta nell'oggetto `lastRequestedOperation` ed eseguendo una `notify()` locale per aggiornare la visualizzazione. La notifica viene intercettata dal controller `AIController` tramite il metodo `update()`, il quale interroga la View tramite `getLastRequestedOperation()` per recuperare i dati dell'operazione. Il controller esegue la propria logica interna tramite `onAIRequest()` e richiama il metodo `requestAIOperation(type, text)` esposto da `AIRequestModel`. A questo punto, il Model imposta il proprio stato interno (`aiState`) su `Processing` e si auto-notifica, propagando l'aggiornamento a tutti gli observer registrati. `AIPanelView` intercetta la notifica `update()`, interroga lo stato tramite `getAIState()`, e, ricevendo la variante `ProcessingState`, mostra immediatamente all'utente uno spinner visivo di caricamento e l'opzione per consentire l'interruzione manuale (R72-F-O, R73-F-O). In parallelo, `AIRequestModel` avvia la richiesta asincrona di elaborazione verso il server richiamando il metodo `requestOperation(type, text)` del componente di infrastruttura `AIServiceProxy`.

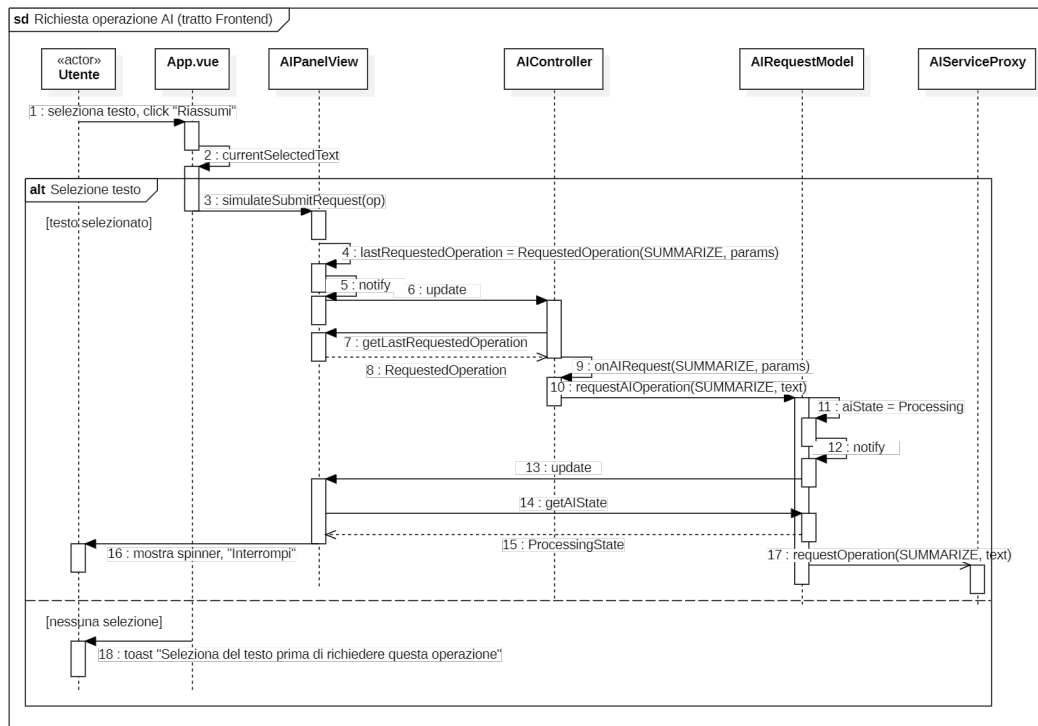


Figura 11: Richiesta operazione AI (tratto Frontend).

1b – Richiesta AI (tratto REST→LLM). AIRouter riceve la richiesta e la inoltra ad AIService, che risolve la strategia corretta tramite la Simple Factory e costruisce un Prompt tramite PromptBuilder. La chiamata attraversa poi il decorator di logging fino all'OpenAIAdapter, che contatta il servizio LLM esterno; la risposta risale la stessa catena fino a AIService, che costruisce la Proposal e la restituisce come AIOperationResponse.

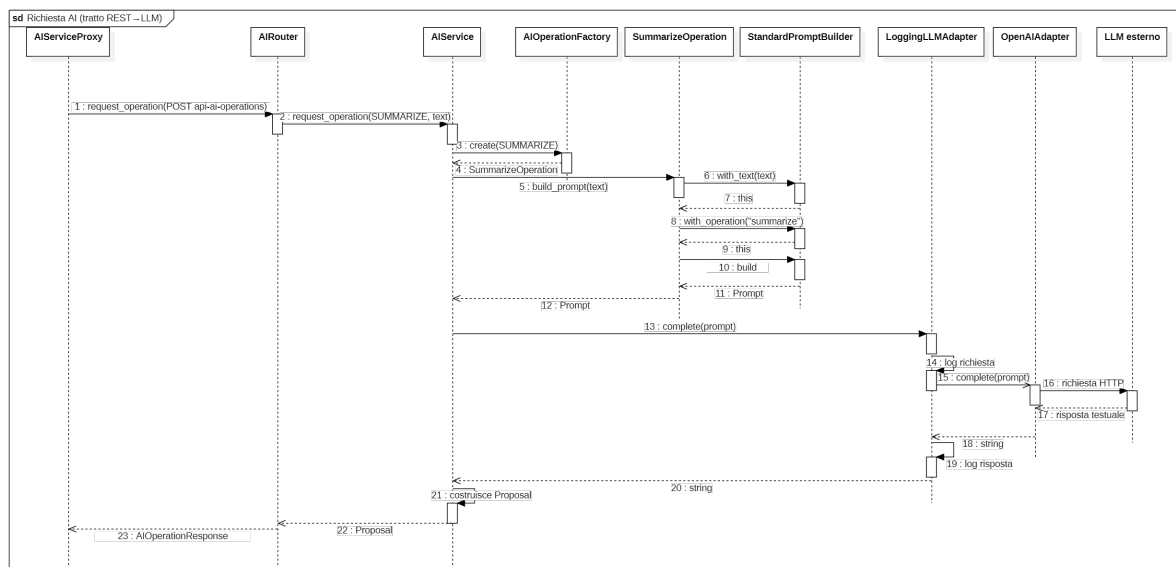


Figura 12: Richiesta AI (tratto REST→LLM): attraversa in un unico flusso i pattern Strategy, Builder, Simple Factory, Decorator e Adapter.

1c – Ricezione proposta AI (tratto Frontend). La risposta HTTP, una volta risolta la Promise di `AIServiceProxy.requestOperation()`, riporta il controllo al Model: `AIRequestModel` transita allo stato `ProposalReadyState(proposal)` e notifica gli observer; `AIPanelView` interroga lo stato aggiornato e mostra all’utente la proposta con le azioni disponibili (Accetta/Rifiuta/Rigenera).

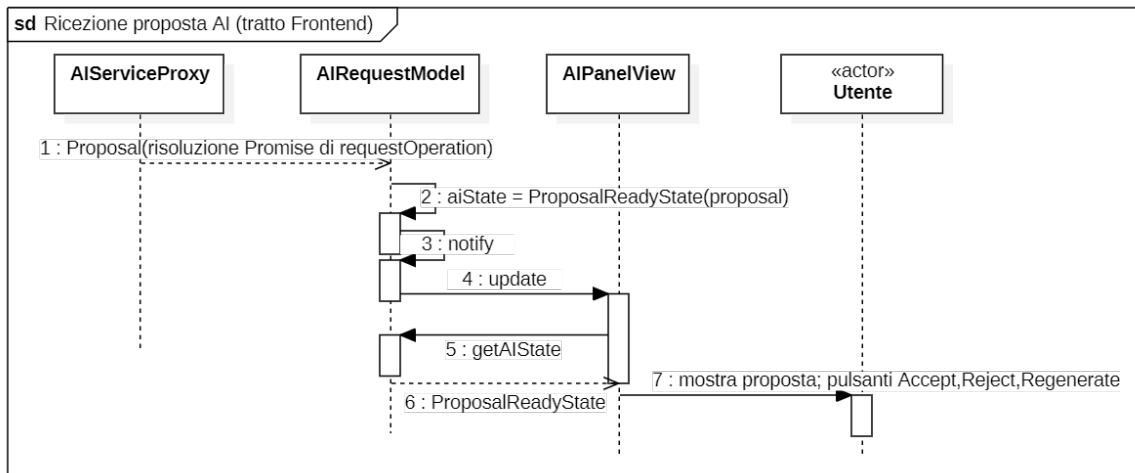


Figura 13: Ricezione proposta AI (tratto Frontend).

7.2 Gestione della proposta AI ed elenco delle operazioni

Accetta proposta AI (riuso Command). Se l’utente accetta la proposta, `AIController` recupera lo stato da `AIRequestModel` (`ProposalReadyState`) e sceglie il comando in base alla presenza del range originario della selezione, tramite un frame `alt`: nel ramo normale (`[range definito]`, praticamente sempre) crea un `ReplaceContentCommand` che sostituisce il testo selezionato con la proposta; nel ramo di rete di sicurezza (`[range non definito]`, es. una selezione persa nel frattempo) crea invece un `InsertTextCommand` che inserisce la proposta in coda al documento. In entrambi i casi il comando è eseguito tramite `NoteModel.executeCommand()`. Il Model torna quindi allo stato `Idle` e il pannello proposta si chiude.

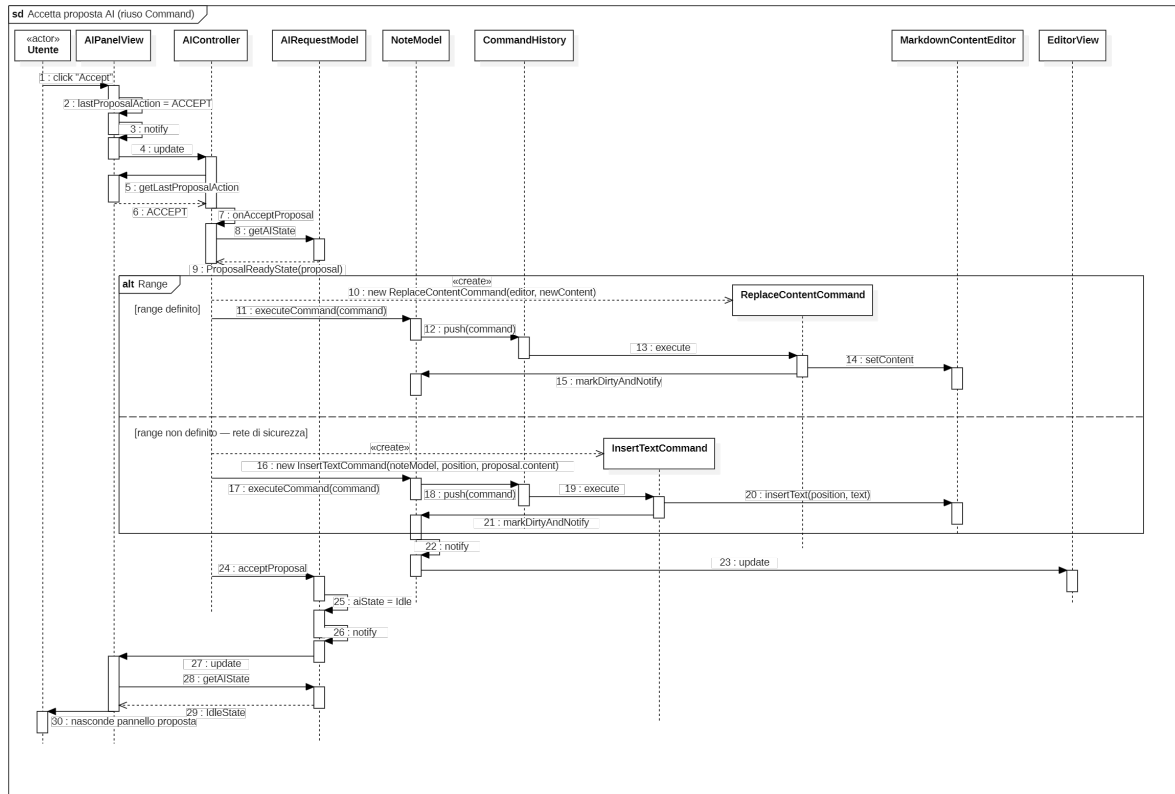


Figura 14: Accetta proposta AI (riuso del pattern Command): `ReplaceContentCommand` nel ramo normale, `InsertTextCommand` come rete di sicurezza.

Rifiuta / Rigenera / Interrompi proposta AI. Le altre tre azioni disponibili sulla proposta condividono lo stesso punto di ingresso (`AIController.update()`) e si distinguono in un unico frame `alt` a tre rami: **Rifiuta** riporta semplicemente il Model a `Idle` senza toccare il documento; **Rigenera** ripete la stessa richiesta memorizzata (`lastRequest`) invocando di nuovo `requestAIOperation()` – non esiste un metodo dedicato di rigenerazione sul Model, si riusa lo stesso flusso della richiesta iniziale (Figura 11); **Interrompi** incrementa un contatore di generazione (`requestGeneration`) prima di tornare a `Idle`, così un’eventuale risposta dell’LLM arrivata in ritardo per la richiesta ormai abbandonata viene riconosciuta come obsoleta e ignorata.

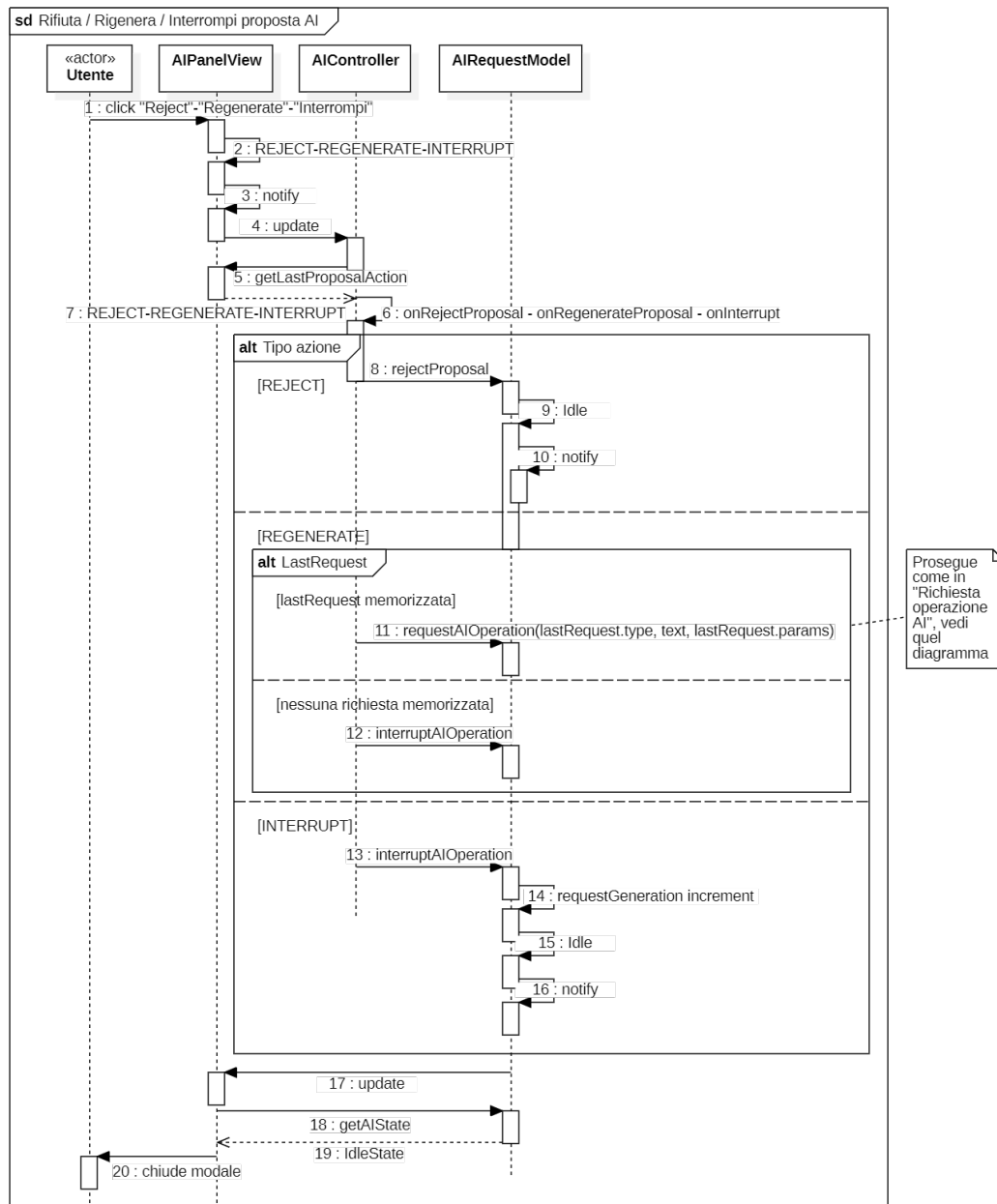


Figura 15: Rifiuta, Rigenera e Interrompi la proposta AI (frame **alt** a tre rami).

Elenco operazioni disponibili (Simple Factory self-registration). L'elenco delle operazioni mostrate nel selettore del Frontend non è hard-coded: `AIPanelView` lo richiede a runtime, risalendo la catena fino ad `AIOperationFactory.available_types()`, che legge il proprio registro di auto-registrazione. Aggiungere una nuova operazione AI non richiede quindi alcuna modifica a questo flusso.

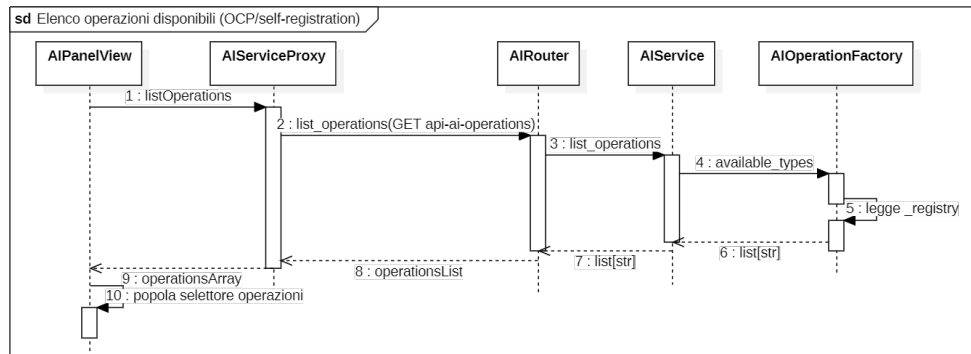


Figura 16: Elenco operazioni disponibili (Simple Factory self-registration).

7.3 Editing del testo e Undo/Redo

I tre diagrammi seguenti mostrano il riuso dello stesso meccanismo Command/CommandHistory per operazioni di editing diverse (formattazione, tabelle) e per il relativo undo/redo.

Formattazione con Undo. La View registra la richiesta dell'utente (es. "Grassetto"), si auto-notifica, il Controller la preleva e crea il **FormatTextCommand** corrispondente, eseguito e archiviato da **CommandHistory**. L'Undo riesegue in ordine inverso l'ultimo comando in pila, ripristinando il contenuto precedente tramite lo stesso oggetto comando.

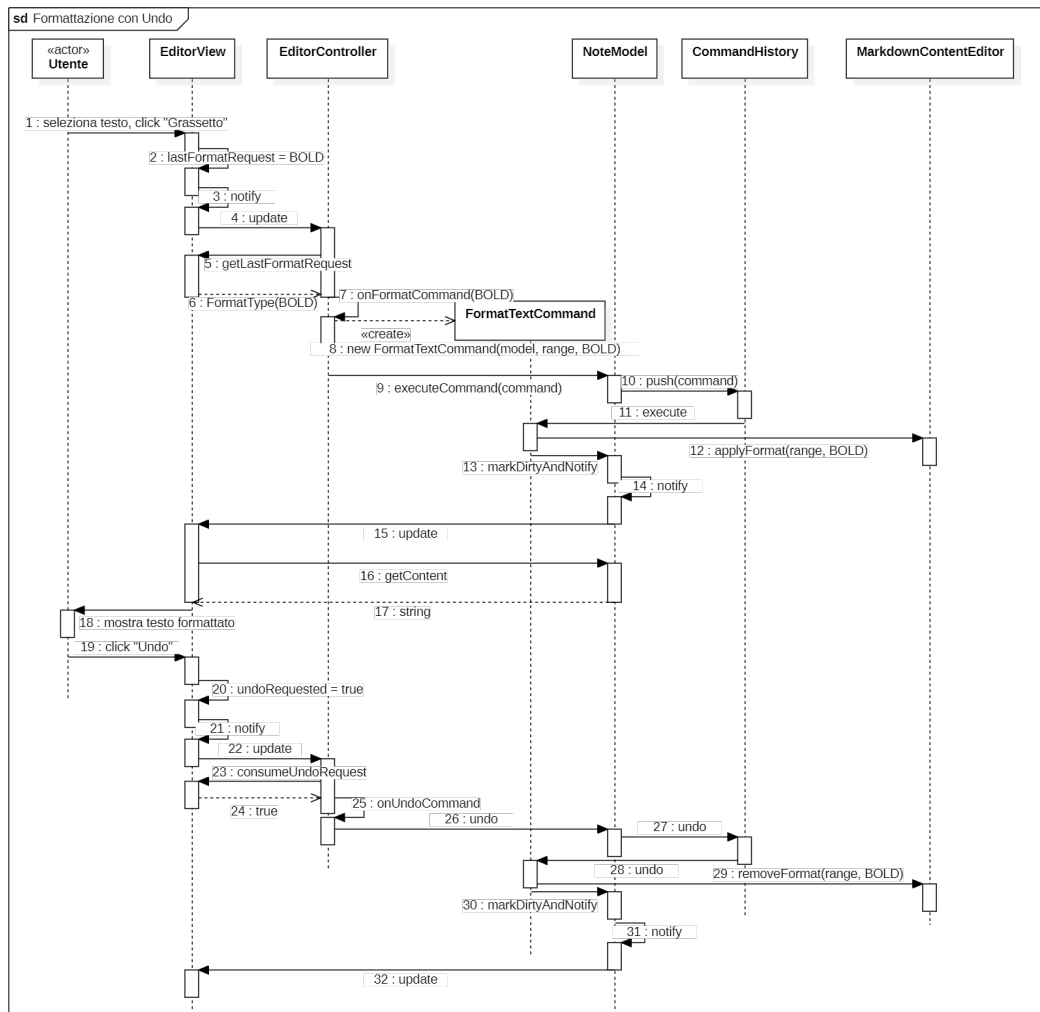


Figura 17: Formattazione di un testo e relativo Undo.

Redo. Continuazione dello scenario precedente: il Redo riesegua l'ultimo comando spostato dalla pila di redo a quella di undo.

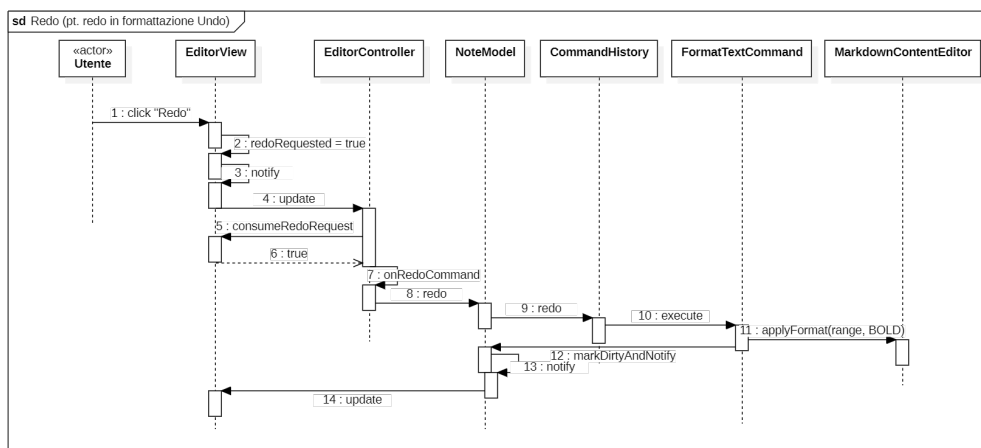


Figura 18: Redo (continuazione di "Formattazione con Undo").

Inserimento tabella con validazione. Il frame **alt** distingue due rami alternativi: se le dimensioni richieste (righe/colonne) non sono valide, **MarkdownContentEditor** solleva **InvalidTableDimensionError**, propagato fino alla View che mostra l'errore senza modificare il documento; se le dimensioni sono valide, la tabella viene inserita e la nota aggiornata normalmente.

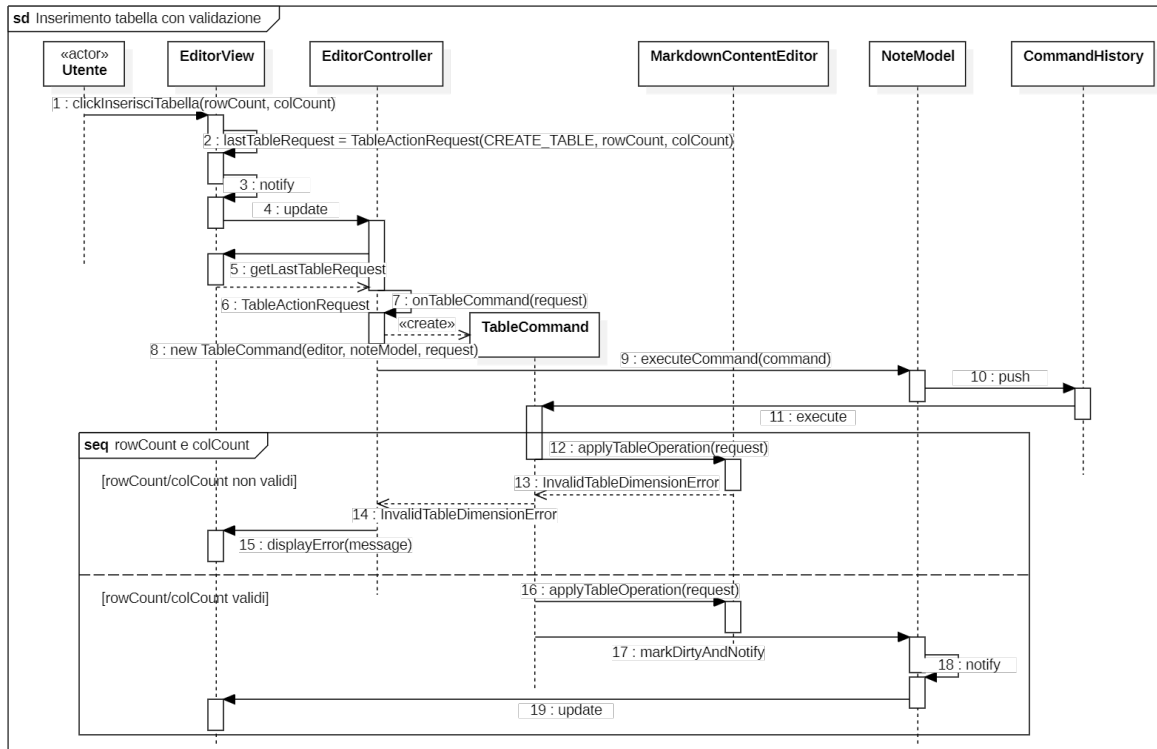


Figura 19: Inserimento di una tabella, con validazione delle dimensioni.

Copia / Taglia / Incolla. Le tre operazioni sugli appunti condividono lo stesso frame **alt** esterno a tre rami, ciascuno con un ulteriore controllo interno sulla presenza di una selezione (Copia, Taglia) o sul permesso di lettura degli appunti concesso dal browser (Incolla): in assenza di selezione o di permesso, l'operazione si conclude con un avviso, senza modificare il documento. Taglia e Incolla, quando vanno a buon fine, riusano **ReplaceContentCommand** – lo stesso comando generico impiegato per raggruppare in un'unica voce di **CommandHistory** le sequenze di digitazione libera – invece di introdurre comandi dedicati.

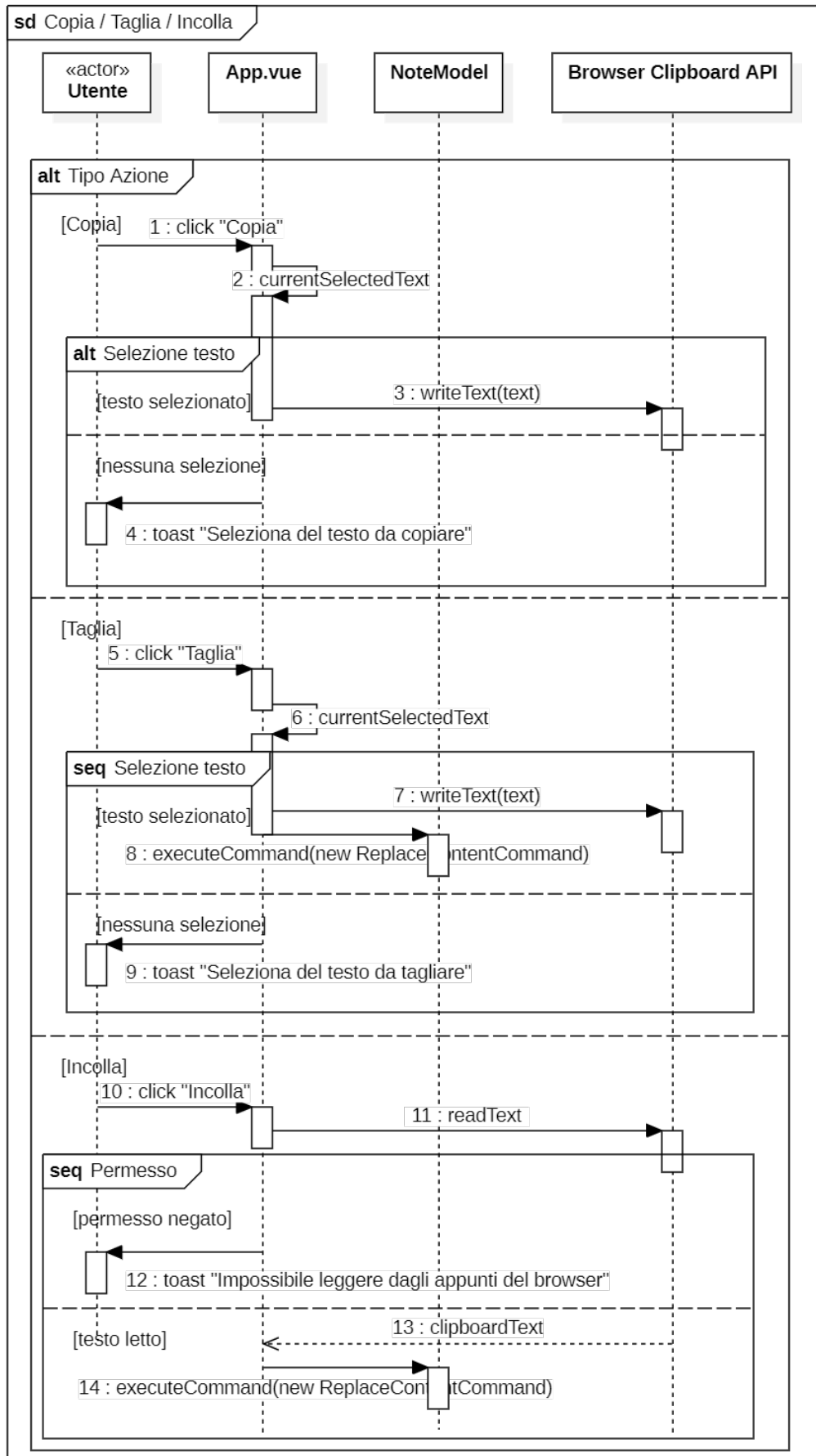


Figura 20: Copia, Taglia e Incolla (frame alt a tre rami).

7.4 Persistenza locale della nota

Salvataggio nota locale. Un primo frame `alt` distingue il salvataggio in base al supporto della File System Access API da parte del browser: se supportata (Chrome, Edge), `NoteServiceProxy` apre il selettore nativo (`showSaveFilePicker`) al primo salvataggio e riusa l'handle ottenuto per i successivi; se non supportata (Firefox, Safari – R5-V-O), ricade su un fallback basato su download (`Blob` + elemento `<a download>`), chiedendo il nome del file una sola volta per nota tramite un prompt nativo e riproponendolo nei salvataggi successivi.

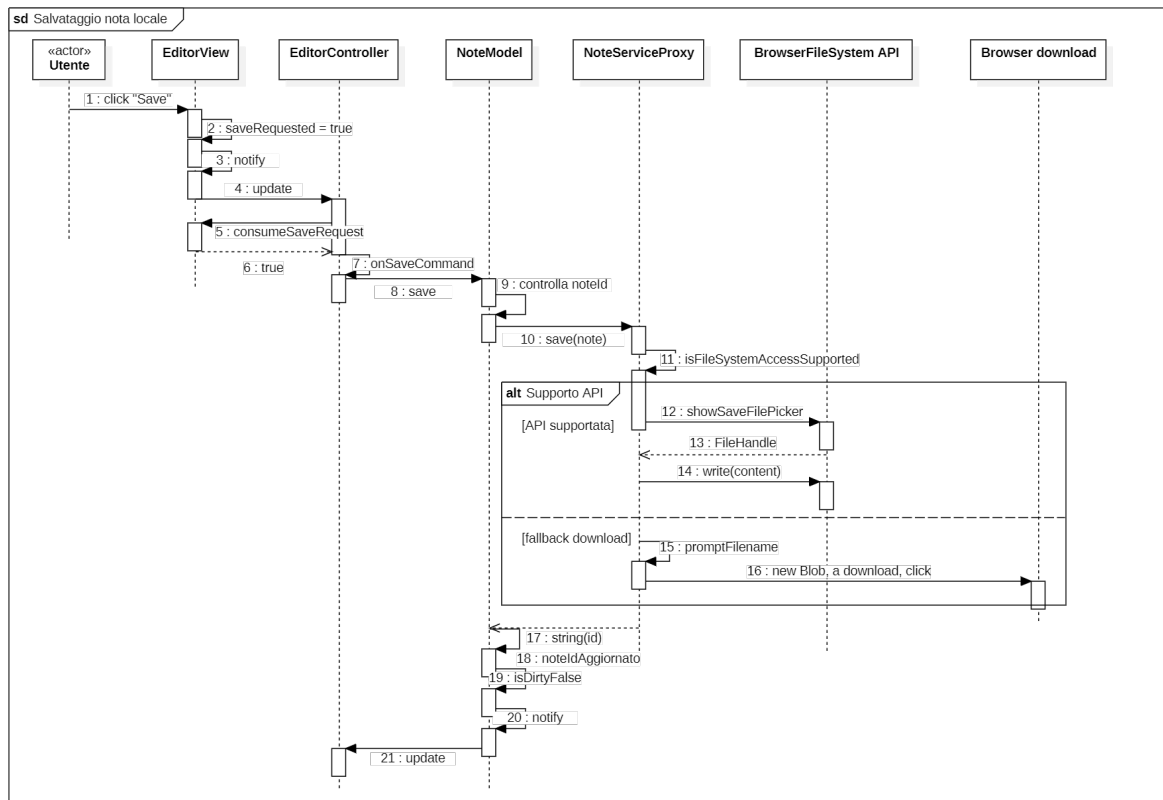


Figura 21: Salvataggio di una nota in locale.

Salvataggio con errore. In caso di errore di I/O durante la scrittura, l'eccezione `NoteIOError` risale da `BrowserFileSystemAPI` fino alla View, che notifica l'utente mantenendo la nota invariata.

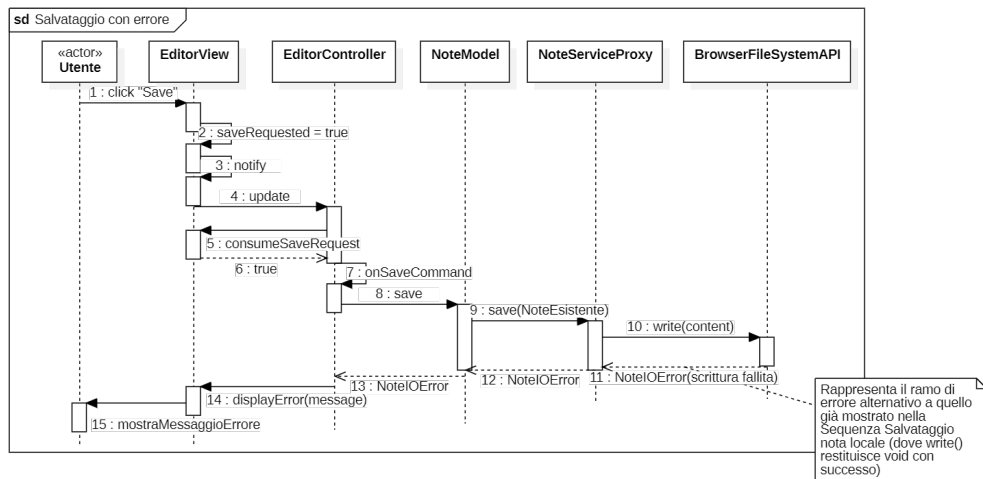


Figura 22: Salvataggio con errore di I/O.

Importazione/Apertura nota. Analogamente al salvataggio, un frame **alt** distingue il supporto della File System Access API: se supportata, l'apertura passa dal selettore nativo del browser (`showOpenFilePicker`), la lettura del contenuto, la ricostruzione dell'oggetto `Note` e l'aggiornamento di `MarkdownContentEditor`; la `CommandHistory` viene azzerata per non permettere un Undo o Redo che attraversino note diverse. Se non supportata (R76-F-O, R5-V-O), il fallback usa un elemento `<input type="file">` nascosto: un frame **alt** interno distingue il caso in cui l'utente sceglie un file da quello in cui annulla il selettore – rilevato tramite il rientro del focus sulla finestra, in assenza di un evento `cancel` affidabile su tutti i browser – che produce un `NoteIOError` con `cancelled=true`.

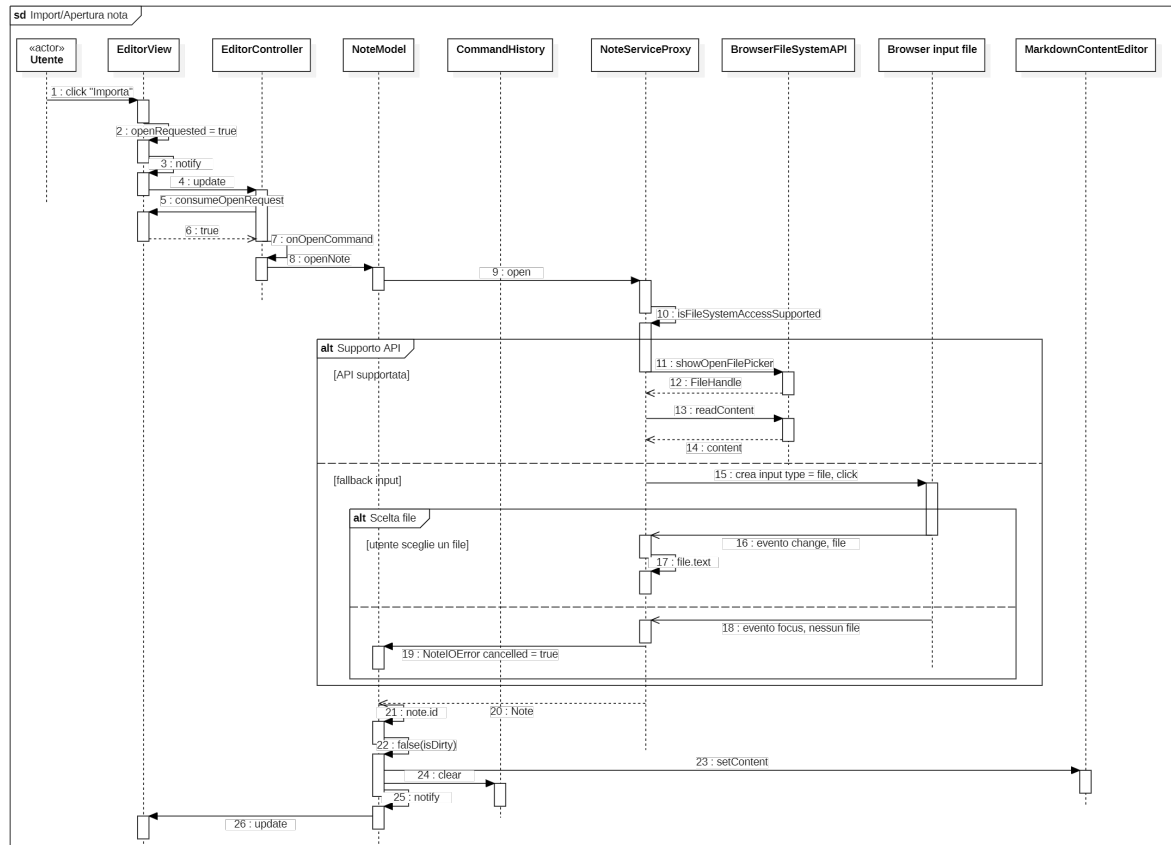


Figura 23: Importazione/Apertura di una nota esistente, con fallback per browser senza File System Access API.

Apertura nota con errore. Speculare a “Salvataggio con errore”: in caso di errore di I/O durante la lettura, l’eccezione `NoteIOError` risale da `BrowserFileSystemAPI` fino alla View. Un frame `alt` distingue il caso in cui l’utente ha semplicemente annullato il selettore nativo (`error.cancelled === true`), mostrato con un tono neutro non allarmante, dal caso di un vero errore di I/O, mostrato come errore.

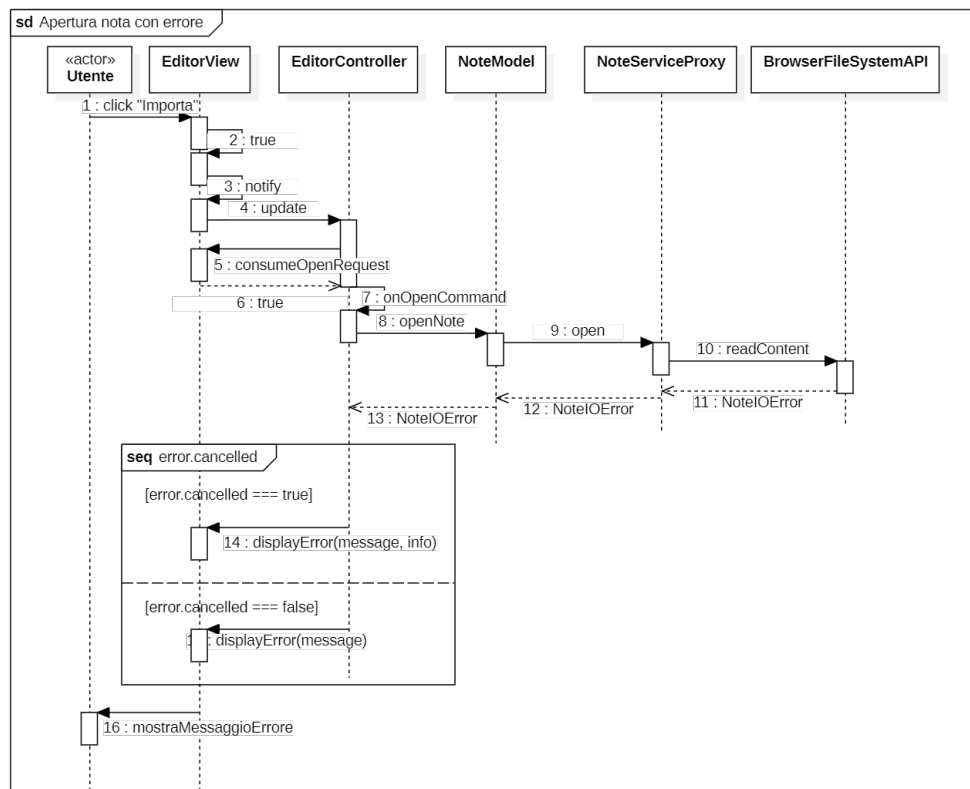


Figura 24: Apertura nota con errore, con distinzione tra annullamento utente ed errore di I/O.

7.5 Esportazione

L'esportazione della nota corrente (R77-F-O) segue un principio di indirezione controllata lato frontend, finalizzato a mantenere un accoppiamento debole tra i moduli. Al click su "Esporta", `App.vue` delega la richiesta a `NoteModel.exportContent()`, il quale instrada la chiamata verso `ExportServiceProxy` solo se il servizio di esportazione è stato correttamente iniettato nel costruttore del Model (in caso contrario, il flusso si interrompe sollevando un'eccezione esplicita). Questa indirezione attraverso il Model, anziché una chiamata diretta della View verso il Proxy di rete, mantiene coerente il pattern di Dependency Injection adottato per tutti gli altri servizi di infrastruttura (come `NoteService` e `AIService`)

Export PDF (Template Method). Il flusso di generazione segue rigorosamente lo scheletro fisso definito dalla classe astratta `Exporter` (pattern *Template Method*):

- Viene invocato il metodo protetto `_prepare_content(content)` per effettuare il parsing iniziale del Markdown e strutturare la nota nel modello intermedio di dominio `Content`;
- Viene invocato il metodo protetto `_convert_format(astContent)`, l'unico passo variabile ridefinito in modo polimorfico dalla sottoclasse concreta `PdfExporter` per la compilazione in byte del file PDF finale.

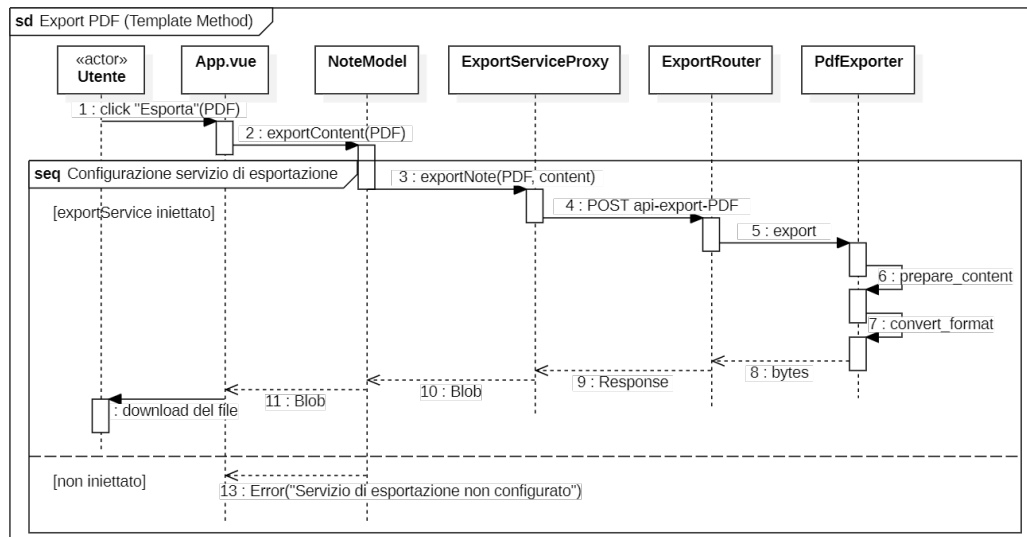


Figura 25: Esportazione di una nota in PDF, dal click in `App.vue` al Template Method sul Backend.

8 Tracciamento e Stato dei Requisiti

Al fine di garantire che l'architettura progettata e le tecnologie scelte soddisfino le funzionalità concordate, si fornisce di seguito la tracciabilità tra le componenti architetturelle descritte in questo documento e i requisiti individuati in [Analisi dei Requisiti](#) (Sezione 4 – Requisiti), a cui si rimanda per la descrizione testuale completa di ciascun requisito.

8.1 Tabella Tracciamento Componenti-Requisiti

Tabella 17: Tracciamento Componenti → Requisiti

Componente	Responsabilità	Requisiti coperti
MarkdownContent-Editor, EditorView/EditorController	Scrittura libera, copia/taglia/incolla, formattazione (grassetto, corsivo, sottolineato, barrato, citazione, intestazione), link, elenchi e tabelle	R1–R42
EditCommand (famiglia), CommandHistory	Annullamento e ripristino delle modifiche	R43-F-O, R44-F-O
EditorView (modalità di visualizzazione)	Visualizzazione esclusiva editor/render _G o affiancata (Split View _G)	R45–R47
AIRouter, AIService, AIOperationFactory, operazioni Strategy	Riassunto, traduzione, riscrittura, Distant Writing e analisi Sei Cappelli tramite LLM	R48, R50, R52, R53, R55, R57, R59, R61, R63, R65, R67
ErrorHandlers, gerarchia AIDomainError	Notifica degli errori di comunicazione con i servizi AI	R49, R51, R54, R56, R58, R60, R62, R64, R66, R68
AIRquestModel, AIPanelView, AIController	Accettazione/rifiuto/rigenerazione della proposta, interruzione manuale, stato di avanzamento continuo	R69–R74
NoteModel, NoteService/NoteServiceProxy	Salvataggio e caricamento della nota in formato Markdown dal file system locale	R75-F-O, R76-F-O
ExportRouter, esportatori per formato (Template Method)	Esportazione della nota in PDF, HTML e JSON	R77-F-O
App.vue	Richiesta di conferma prima di chiudere/navigare fuori dall'applicazione con modifiche non salvate; apertura dei link dell'anteprima in una nuova scheda o segnalazione di URL non valido	R87-F-O, R88-F-O
Continua nella pagina successiva...		

Tabella 17 – Continua dalla pagina precedente

Componente	Responsabilità	Requisiti coperti
Suite di test automatici, pipeline CI	Copertura di test ($\geq 80\%$ sui moduli principali), test di integrazione con i servizi LLM, gestione non bloccante degli errori	R1-Q-O, R2-Q-O, R3-Q-O, R10-Q-O, R11-Q-O
python-dotenv, OpenAIAdapter	Gestione delle credenziali di accesso al servizio LLM esclusivamente lato Backend, mai esposte al client o nei log	R8-Q-O
Architettura Client-Server, adapter LLM	Esecuzione via browser, standard API OpenAI, singolo modello LLM per richiesta, repository pubblico	R1-V-O, R2-V-O, R3-V-O, R5-V-O, R6-V-O, R7-V-O
Scelta libera del framework Frontend	Vue.js adottato senza vincolo imposto dal capitolato	R11-V-O
Docker Compose, Dockerfile	Consegna come archivio distribuibile con configurazione Docker, senza deploy su infrastruttura server esterna	R12-V-O
Rendering reattivo Vue.js, stato AIRequestState	Aggiornamento anteprima entro 1s (anche per note estese), tempo di avvio dell'applicazione, indicatore di caricamento continuo	R1-P-O, R2-P-O, R3-P-O, R4-P-O

I requisiti opzionali R4-V-Op e R78–R86 (persistenza remota, gestione utenti) non compaiono in questa tabella poiché il gruppo ha deciso di non implementarli in questa fase (si veda Sezione 7.2 per lo stato dettagliato e la motivazione): non esiste quindi alcun componente architetturale a cui tracciarli.

Nota: R8-V-O, R9-V-O e R10-V-O (obbligo di produrre rispettivamente Specifica Tecnica, Piano di Qualifica e Manuale Utente) sono requisiti di natura documentale, soddisfatti dall'esistenza dei documenti stessi: non è quindi individuabile un componente architetturale a cui tracciarli, e non compaiono nella tabella precedente.

8.2 Stato di Soddisfacimento dei Requisiti

La tabella seguente riassume, per macro-gruppo, lo stato di implementazione dei requisiti funzionali, di qualità, di vincolo e prestazionali rispetto all'architettura descritta in questo documento. I gruppi ricalcano la struttura di [Analisi dei Requisiti](#), Sezione 4; il numero di requisiti obbligatori (100), desiderabili (5) e opzionali (15) è riepilogato nello stesso documento, Sezione 6 – Riepilogo.

I requisiti opzionali relativi alla persistenza remota e alla gestione utenti (R4-V-Op, R7-Q-Op, R8-Q-Op, R78–R86) sono stati valutati e volutamente esclusi dallo scope_G della release corrente: il gruppo ha scelto di concentrare le risorse disponibili sul consolidamento e sulla qualità dei requisiti obbligatori. Sono pertanto riportati con stato *Non implementato*.

Tabella 18: Stato dei Requisiti

Codici	Descrizione	Tipo	Stato
R1–R4	Scrittura, copia, taglia, incolla di testo nell’editor.	Obbligatorio	Soddisfatto
R5–R28	Attivazione / applicazione / disattivazione / rimozione delle formattazioni grassetto, corsivo, sottolineato, barrato, citazione e intestazione.	Obbligatorio	Soddisfatto
R29–R31	Inserimento, modifica e rimozione di link ipertestuali.	Obbligatorio	Soddisfatto
R32–R35	Inserimento, modifica, disattivazione e rimozione di elenchi puntati e numerati.	Obbligatorio	Soddisfatto
R36–R42	Inserimento e modifica di tabelle Markdown, con validazione dei parametri.	Obbligatorio	Soddisfatto
R43, R44	Annullamento (Undo) e ripristino (Redo) dell’ultima modifica.	Obbligatorio	Soddisfatto
R45–R47	Visualizzazione esclusiva editor, render o affiancata (Split View).	Obbligatorio	Soddisfatto
R48, R49	Generazione riassunto tramite LLM e notifica di errore.	Obbligatorio	Soddisfatto
R50, R51	Traduzione (Inglese, Francese, Spagnolo) tramite LLM e notifica di errore.	Obbligatorio	Soddisfatto
R52	Traduzione in Tedesco come estensione facoltativa.	Desiderabile	Soddisfatto
R53, R54	Riscrittura del testo tramite LLM e notifica di errore.	Obbligatorio	Soddisfatto
R55, R56	Distant Writing da prompt utente e notifica di errore.	Obbligatorio	Soddisfatto
R57–R68	Analisi critica secondo le sei prospettive dei Cappelli di De Bono e relative notifiche di errore.	Obbligatorio	Soddisfatto
R69–R71	Accettazione, rifiuto e rigenerazione della proposta AI.	Obbligatorio	Soddisfatto
R72, R73	Visualizzazione continua dello stato di avanzamento e interruzione manuale della richiesta AI.	Obbligatorio	Soddisfatto
R74	Visualizzazione della proposta AI in un modale dedicato.	Obbligatorio	Soddisfatto
Continua nella pagina successiva...			

Tabella 18 – Continua dalla pagina precedente

Codici	Descrizione	Tipo	Stato
R75, R76	Salvataggio e caricamento della nota in formato Markdown su file system locale.	Obbligatorio	Soddisfatto
R77	Esportazione della nota in PDF, HTML e JSON.	Obbligatorio	Soddisfatto
R87, R88	Conferma prima di uscire con modifiche non salvate; apertura dei link dell'anteprima (nuova scheda o segnalazione URL non valido).	Obbligatorio	Soddisfatto
R78–R86	Creazione/eliminazione nota, registrazione/autenticazione utente, gestione note remote.	Opzionale	Non implementato
R1-Q-O, R2-Q-O, R3-Q-O	Copertura di test ($\geq 80\%$ sui moduli principali), versionamento Git, test di integrazione con i servizi LLM.	Obbligatorio	Soddisfatto
R8-Q-O, R10-Q-O, R11-Q-O	Gestione delle credenziali LLM esclusivamente lato Backend; continuità dell'editing locale e gestione non bloccante degli errori in caso di indisponibilità dei servizi esterni.	Obbligatorio	Soddisfatto
R4-Q-Op, R5-Q-Op, R9-Q-Op	Test di integrazione, autenticazione e hashing delle password del modulo opzionale di persistenza remota.	Opzionale	Non implementato
R6-Q-D	Adozione di una licenza open source permissiva.	Desiderabile	Soddisfatto
R7-Q-D	Criteri minimi di usabilità dell'editor (contrasto, leggibilità, focus da tastiera).	Desiderabile	Soddisfatto
R12-Q-D	Architettura modulare che consenta la sostituzione del provider LLM senza impatti significativi.	Desiderabile	Soddisfatto
R13-Q-D	Assenza di dipendenza da funzionalità non standard dei browser.	Desiderabile	Soddisfatto
R1-V-O – R3-V-O, R5-V-O – R7-V-O	Esecuzione via browser standard, interazione con i modelli LLM tramite API compatibile OpenAI, singolo modello per richiesta, repository pubblico.	Obbligatorio	Soddisfatto
R8-V-O – R10-V-O	Produzione di Specifica Tecnica, Piano di Qualifica e Manuale Utente.	Obbligatorio	Soddisfatto
R11-V-O	Libertà di scelta del framework Frontend (Vue.js).	Obbligatorio	Soddisfatto
Continua nella pagina successiva...			

Tabella 18 – Continua dalla pagina precedente

Codici	Descrizione	Tipo	Stato
R12-V-O	Consegna come archivio Docker distribuibile, senza deploy su infrastruttura server esterna.	Obbligatorio	Soddisfatto
R4-V-Op	Persistenza remota tramite un database server-side.	Opzionale	Non implementato
R13-V-Op	Predisposizione per il deploy su server esterno, in aggiunta all'archivio Docker.	Opzionale	Non implementato
R1-P-O	Aggiornamento dell'anteprima Markdown entro 1 secondo dalla modifica.	Obbligatorio	Soddisfatto
R2-P-O	Visualizzazione continua e reattiva dello stato di caricamento durante l'elaborazione AI.	Obbligatorio	Soddisfatto
R3-P-O	Tempo di avvio dell'applicazione entro 3 secondi in condizioni di rete standard.	Obbligatorio	Soddisfatto
R4-P-O	Rendering dell'anteprima senza degrado percettibile anche per note estese (>10.000 caratteri).	Obbligatorio	Soddisfatto
R5-P-O	Timeout di sicurezza per le richieste AI (soglia elevata, non percepibile in condizioni normali); unico controllo effettivo sui tempi affidato all'interruzione manuale (R73-F-O).	Obbligatorio	Soddisfatto
R6-P-Op	Caricamento della lista delle note remote entro 3 secondi.	Opzionale	Non implementato

9 Requisiti di Sistema

Questa sezione esplicita i requisiti hardware, software e di rete assunti dall'architettura descritta nel documento, a cui i requisiti prestazionali del capitolo 5 – Requisiti dell'Analisi dei Requisiti (in particolare R1-P-O, R3-P-O, R4-P-O) fanno riferimento senza definirli esplicitamente.

9.1 Hardware

Trattandosi di un'applicazione web in cui l'elaborazione AI è interamente delegata al servizio LLM esterno (Sezione 3.4), il carico computazionale lato client si limita al rendering dell'editor e dell'anteprima Markdown; non sono pertanto richieste risorse hardware dedicate oltre a quelle di un dispositivo desktop o laptop di fascia standard in commercio negli ultimi 3–4 anni.

- **CPU:** processore dual-core con clock ≥ 1.6 GHz o equivalente, sufficiente a garantire i tempi di avvio (R3-P-O) e di aggiornamento dell'anteprima (R1-P-O, R4-P-O) dichiarati.
- **RAM:** 4 GB minimi consigliati per l'esecuzione del browser con una singola scheda dedicata all'applicazione, tenendo conto dell'overhead di CodeMirror_G e del parsing Markdown lato client.
- **Storage:** nessun requisito significativo; il sistema non mantiene alcuna persistenza locale automatica oltre ai singoli file `.md` esplicitamente salvati dall'utente (R75-F-O), la cui dimensione è trascurabile rispetto alla capacità di un qualsiasi dispositivo moderno.
- **Lato server:** nessun requisito hardware specifico è imposto dal gruppo, poiché l'infrastruttura di calcolo per l'inferenza LLM è gestita e messa a disposizione dall'azienda proponente (R3-V-O); il Backend Second Brain richiede solo le risorse minime per l'esecuzione di un servizio FastAPI containerizzato (Sezione 3.2).

Il degrado prestazionale sotto queste soglie non è stato misurato sistematicamente dal gruppo: i valori dichiarati in R1-P-O, R3-P-O e R4-P-O sono da intendersi validi per dispositivi conformi ai requisiti sopra indicati, mentre su hardware inferiore l'unica garanzia esplicita rimane l'assenza di blocchi o crash dell'editor (R10-Q-O).

9.2 Software

- **Browser:** come vincolato da R5-V-O, il sistema deve essere eseguito su Chrome 151+, Edge 150+, Firefox 153+ o Safari 26+.
- **Sistema operativo:** nessun vincolo diretto oltre alla disponibilità di uno dei browser sopra indicati; l'applicazione è stata verificata su Windows, macOS e Linux.
- **Dipendenza da API browser-specifiche:** le funzionalità di salvataggio e apertura locale della nota (R75-F-O, R76-F-O) si appoggiano preferibilmente alla *File System Access API* del browser (`showSaveFilePicker`, `showOpenFilePicker`, Sezione 3.3), nativa su Chrome ed Edge. Su Firefox e Safari, dove questa API non è disponibile, `NoteServiceProxy` rileva l'assenza a runtime e ricade su un fallback equivalente basato su API standard (download tramite `Blob/``<a download>` per il salvataggio, `<input type="file">` per l'apertura), garantendo così la compatibilità con l'intero parco browser richiesto da R5-V-O senza violare R13-Q-D (nessuna dipendenza da funzionalità non standard esposta al resto del sistema, dato che il contratto `NoteService` resta invariato in entrambi i casi).

9.3 Network

- **Editing locale:** le funzionalità di scrittura, formattazione e cronologia delle modifiche (R1–R47) non richiedono connettività di rete, essendo interamente gestite lato client (R10-Q-O).
- **Operazioni AI:** richiedono una connessione di rete attiva tra Frontend e Backend e tra Backend e servizio LLM esterno; la comunicazione avviene su protocollo HTTPS (Sezione 3.4). Non è imposto un requisito esplicito di banda minima. Coerentemente con R5-P-O, il sistema non impone un limite di prodotto percepibile sulla durata delle richieste AI, affidando all'utente – tramite l'interruzione manuale (R73-F-O) – l'unico controllo effettivo sui tempi di attesa; a protezione da connessioni di rete bloccate è comunque applicato un timeout tecnico di sicurezza con soglia elevata (indicativamente 15 minuti), pensato per non essere raggiunto in condizioni di utilizzo normale.
- **Persistenza remota (opzionale, non implementata):** qualora in futuro venisse realizzato il modulo opzionale di persistenza remota (R4-V-Op), andrebbe definito un requisito di rete dedicato per il caricamento della lista delle note (R6-P-Op); allo stato attuale non è applicabile.